SIGNOMIAL PROGRAMMING FOR LIFTING-LINE ANALYSIS AND OPTIMIZATION

A THESIS

Presented to the Department of Mechanical and Aerospace Engineering

California State University, Long Beach

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Aerospace Engineering

Committee Members:

Cody J. Karcher, Ph.D. (Chair)
Ehsan Madadi, Ph.D.
Jingyi Zeng, Ph.D.

College Designee:

Kerop Janoyan, Ph.D.

By Michael Ken Shoda

B.S., 2023, University of California, Irvine

May 2026

ABSTRACT

The Geometric Programming (GP) and Signomial Programming (SP) are categorizations of problems with rapid solution methods. GP and SP are widely applicable across problems of interest in aircraft design, having success in various Multidisciplinary Optimization (MDO) works. With algebraic manipulation and transformation techniques, many aircraft design models can be changed into GP and SP compatible forms, allowing complex design problems to be solved with reduced computational cost. Previous works have shown GP and SP to be applicable across a number of aircraft design problems but have relied on low fidelity methods to approximate induced drag. This work develops a novel Signomial Programming-compatible optimization methodology to the Lifting-Line Theory (LLT) with the objective of minimizing induced drag on a given wing configuration. A Python-based model is created using LLT equations as constraints, derived using a discrete panel method. With an SP compatible formulation, the model can be solved for the potential jump distribution that minimizes induced drag and satisfies geometric and aerodynamic constraints. The constructed novel SP-compatible model is validated against a more traditional LLT model that assembles and solves an Aerodynamic Influence Coefficient (AIC) matrix. Results from both approaches converge to the classical optimal lift distribution for a flat wake, and demonstrate expected behavior and convergence for other various wake shapes. The results establish a foundation for integrating higher-fidelity aerodynamic modeling into SP aircraft design frameworks.

ACKNOWLEDGMENTS

I would like to thank my advisor, Dr. Cody Karcher, for his guidance and support throughout the entire research and writing process. I am sincerely grateful for his knowledge, enthusiasm, and understanding, without which this project would not have been possible.

I would also like to thank the committee members, Dr. Jingyi Zeng and Dr. Ehsan Madadi, for their time and effort in reviewing this work.

I am grateful for the connections and friendships formed with my peers at CSULB. I would especially like to thank David Avila for his insight as a fellow graduate student and assistance in formatting this thesis.

Finally, I would like to thank my family for their continuous support and encouragement, and for continuing to inspire me to this day.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1

NUMERICAL OPTIMIZATION IN AIRCRAFT DESIGN

1.1 Aircraft Design Evolution and Optimization

With the rapid technological advancements throughout the last few decades, aircraft design has evolved from hand calculations and empirical data collection to system-wide computational design and simulation. Design has progressed from simply producing viable designs into thorough optimization for fulfilling design requirements and mission objectives. Traditionally, the process is iterative, using known relationships and assumptions from previous designs and data to quickly create and modify designs. However, the inherently multidisciplinary nature of aircraft design naturally leads to complex problems with numerous design considerations and constraints. Aircraft design requires analysis of systems of multidisciplinary models and how their interactions affect the overall configuration. Small changes to one subsection can have long-reaching effects on the end results, requiring careful study into design tradeoffs and adjustments in the optimization process. Numerical optimization is a key tool for its applicability to the systems of models necessary for modern aircraft design, identifying high performance designs for a given design problem.

1.1.1 Numerical Optimization

A numerical optimization problem consists of an objective function to minimize, constraints that must be satisfied for a feasible design, and design variables that are adjusted to yield an optimal solution. Aircraft design objectives are in physical form, typically including traits such as vehicle weight and range. Constraints are added as equalities or inequalities to enforce physical and practical design constraints [1]. While a constrained optimization problem can be written as an equivalent unconstrained Lagrangian function, the constrained problem can be solved more efficiently [2].

The numerical optimization design cycle ideally takes a candidate design and executes the mathematics to provide performance data. The design engineer analyzes the data and makes

changes or refinements to the design as necessary. The new design is passed once more through the mathematics until performance results are satisfactory and the corresponding design is finalized. Expanding the range of analysis for design spaces and constraints results in more efficient design solutions. Executing the iterative process with computational technology makes the process more efficient and reduces human effort in repetitive calculations. Aircraft design is a fundamentally human-driven process; applying the proven numerical optimization processes to aircraft design provides powerful tools for exploring large design spaces and identifying high performance solutions to problems of interest.

1.2 Multidisciplinary Design Optimization

Multidisciplinary Design Optimization (MDO) methods have been a topic of interest in aircraft design optimization due to their potential in solving large complex systems, providing the framework to apply numerical optimization to aircraft design. System problems are broken down into subproblems that are solved with a particular disciplinary analysis model. Analysis models serve as computational mappings from a vector of inputs (i.e. freestream velocity and chord length) to a vector of outputs (i.e. lift and drag) and range from simple equations to sophisticated software packages, covering a wide variety of disciplines.

Karcher [1] describes the analysis model with three main components. First, the preprocessor translates the inputs into values that can be used in the analysis model, often involving geometric translation. The second component is the mathematical representation, which is specific to the discipline of the model. Lastly, there is the computational software that calculates the numerical solution. The resulting outputs can be quickly analyzed for design viability, and the process repeated for conditions adjusted as desired. After solving, coupled design variables are passed between the various analysis models [3].

By nature, aircraft design systems commonly use several hundred design variables and constraints, which result in higher order and nonlinear systems. For example, an aircraft design

system may involve sizing for wings, tails, and engine components, mission requirements for payload weight and range, and a fuel minimization objective. The most effective optimizations and designs for this type of system require consideration of the interactions between each of the components. Multiple analysis models are required for full system analysis, historically used in black-box formats for necessary practical convenience. Interactions between black-box models make the resulting problem inherently multidisciplinary, motivating development into solving methods [4].

The MDO field has numerous architectures that unify numerical optimization and analysis models into a singular framework. Primarily, the difficulty of implementation, formulation, and optimization are the main points of comparison, along with the number of analytical tools available to integrate [3]. MDO architectures are either monolithic or distributed. Monolithic architectures solve a single optimization problem, while the distributed architecture divides the problem into multiple problems with variable and constraint subsets [4].

Depending on the problem, certain architectures are best suited for solving, requiring thought into determining the most viable options for a given problem. Choosing a highly compatible architecture can greatly reduce the computational time, particularly important for medium and large scale problem architectures which struggle to converge quickly [4]. Solving such large and complex design problems can be impractically expensive and is not guaranteed to reach a global optimum in black-box analysis [5].

1.3 Simultaneous Analysis aNd Design (SAND)

The architecture most relevant to this work is Simultaneous Analysis aNd Design (SAND), an example of the All-at-Once (AAO) architecture [4]. SAND and AAO are both monolithic. The defining characteristic of AAO and SAND is direct implementation of design variables and state variables as optimization variables and governing equations as constraints. This structure allows the simultaneous optimization of the design and analysis of the subdiscipline methods.

SAND differs from AAO by replacing the coupled variable targets from the AAO formulation with the coupled variable responses, decreasing the size of the problem [4]. Eliminating specific discipline analysis allows the optimizer to analyze infeasible regions of the overall problem with respect to the constraints. SAND can also be used for both single and multidisciplinary system optimizations.

The ability to integrate numerous subsystem components into a system-wide model optimization makes the SAND architecture potentially quite effective for aircraft design problems. Among the main types of architectures, SAND is the most efficient for optimization. However, it is the most difficult to implement and has the largest number of variables [3]. A major disadvantage of SAND is the requirement that all state variables and equations, including those within specific disciplines, must be included in the optimization problem. Problems can become quite large, thereby increasing the difficulty of convergence, model instability, and the likelihood of arriving at infeasible designs. The second issue is formulating problems in the correct structure. Black-box type models are only able to be implemented if state variables and residuals are available to the optimizer, requiring code modifications to existing models or entirely new model formulations [4]. Nevertheless, aircraft design equations can often be manipulated into the proper form for implementation, as shown later in this work. Developments in SAND methods have discovered a natural connection to a Geometric Programming-centered approach [6].

1.4 Geometric Programming

Geometric Programming (GP) was first introduced by Duffin, Peterson, and Zener [7], and first applied to aircraft design by Hoburg and Abbeel [6] and Torenbeek [8]. GP has since been used in urban air mobility design [9], electrical engineering applications [10, 11, 12], turbine design [13], biotechnological systems [14, 15], and biochemical applications [16, 17].

The defining trait of geometric programs is their convexity after a logarithmic change of variables and a logarithmic transformation of the objective and constraint functions, known as a

log-log transformation. Upon transformation, convex optimization is used to quickly solve for a global optimum [6, 18].

Geometric Programming is built on two mathematical forms of expression—monomials and posynomials. Monomial functions are defined as [18]:

$$m(x) = c \prod_{j=1}^{n} x_j^{a_j} \tag{1.1}$$

where exponents a_j are real numbers, the coefficient c is a positive real number, and variables x_j are positive real numbers. Posynomial functions are sums of monomial functions [5]:

$$p(x) = \sum_{k=1}^{K} c_k \prod_{j=1}^{n} x_j^{a_{jk}} \tag{1.2}$$

under the same conditions—exponents a_j are real numbers, the coefficient c is a positive real number, and variables x_j are positive real numbers. Using these definitions, a GP problem is defined as a posynomial objective subject to monomial and posynomial constraints. In GP models, monomial constraints are written as equalities, and posynomials written as inequalities [18].

$$\begin{aligned} \text{minimize} \quad & p_0(x) \\ \text{subject to} \quad & p_i(x) \leq 1, \quad i = 1, \ldots, N \\ & m_j(x) = 1, \quad j = 1, \ldots, N \end{aligned} \tag{1.3}$$

Expressions that are written in or able to be changed to the form of Equation 1.3 are called GP compatible. As noted previously, GP compatible formulations become convex via log-log transformation. The log transformation is demonstrated using some standard variable x_i and the logspace variable y_i:

$$y_i = \log x_i \tag{1.4}$$

with the transformed GP formulation becoming:

$$\begin{aligned} \text{minimize} \quad & \log p_0(e^y) \\ \text{subject to} \quad & \log p_i(e^y) \leq 0, \quad i = 1, \ldots, N \\ & \log m_j(e^y) = 0, \quad j = 1, \ldots, N \end{aligned} \tag{1.5}$$

The transformed monomial constraint is written out (1.6) and expanded (1.7):

$$\log m(e^y) = \log(c e^{a_1 y_1} e^{a_2 y_2} \ldots e^{a_n y_n}) \tag{1.6}$$

$$\log m(e^y) = \log c + \log e^{a_1 y_1} + \log e^{a_2 y_2} + \cdots + \log e^{a_n y_n} \tag{1.7}$$

and then simplified as:

$$\log m(e^y) = \log c + a_1 y_1 + a_2 y_2 + \cdots + a_n y_n \tag{1.8}$$

which is an affine function in y. Using similar simplifications, the transformed posynomial constraint becomes:

$$\log p(e^y) = \log\left(\sum_{k=1}^{K} c_k \prod_{j=1}^{N} e^{a_{ik} y_i}\right) \tag{1.9}$$

which is the known convex log-sum-exp function. With a convex objective and feasible set, Equation 1.5 is defined as the logarithmic form for a convex optimization problem [1].

Once the functions are convex, the global optimum can be found using efficient computing methods without need for starting points or initial guesses [19]. Interior point method solvers such as CVXOPT [20] are most commonly used, and on an ordinary computer are able to solve a problem with 1000 variables and 10000 constraints in under a minute [18]. Convexity is crucial to GP formulations, as methods for nonlinear problems that are not known to be convex lose the global optimality and solving efficiency that merit using GP [18].

Solvers also determine whether all the constraints of the GP problem are feasible. Infeasible problems are flagged for troubleshooting and adjustments, such as constraint relaxations. Once an objective is solved, the constraints can also be adjusted for parameter tradeoff and sensitivity analysis. Constraints and objectives can be tightened or relaxed to view their effects on the model output characteristics and potentially change the feasibility of a problem [18].

While powerful, the GP compatible form is fairly restrictive. Design variables must be positive and nonzero, and constraints must be greater than zero. The posynomial functions must be upper bounded to make the problem convex. Certain limitations of GP can be avoided with algebraic manipulation. In some cases, negative values can be addressed with an additive shift or by optimizing the absolute values. The required posynomial objective and upper bounded constraints are the most restrictive conditions, as trigonometric (i.e. $\sin(x)$ and $\cos(x)$), exponential (e^x), and logarithmic ($\log(x)$) functions are incompatible with the GP formulation. While these functions can be expanded as Taylor series to become GP-compatible posynomials, the number of expandable terms is limited. For example, the second term of the $\sin(x)$ and $\cos(x)$ expansions are not posynomials [6, 1].

1.5 Signomial Programming

The mathematical rigidity of the GP definition can become quite limiting for certain applications of interest despite the various workaround methods. Signomial Programming is an expansion of the Geometric Programming structure that allows for a wider range of formulations at the cost of convexity and global optimality. Regardless, the expanded application potential beyond solely GP compatible formulations has been demonstrated to be more advantageous in certain circumstances. SP was first applied to aircraft design by Kirschen [5], York [21, 22], and Karcher [1]. SP use has been expanded to other aerospace design applications [23, 24, 25, 26, 27].

A signomial function is defined as the difference between two posynomial functions. The

standard form is [5]:

$$s(x) = p(x) - n(x) = \sum_{k=1}^{K} c_k \prod_{j=1}^{n} x_j^{a_{jk}} - \sum_{p=1}^{P} d_p \prod_{j=1}^{n} x_j^{g_{jk}} \tag{1.10}$$

where $s(x)$ is the signomial function, and $p(x)$ and $n(x)$ are both posynomials. Notably, SP differs from GP in that signomials have both positive and negative posynomials: $p(x)$ consists of all positive coefficient monomials and $n(x)$ consists of all negative coefficient monomials. The term 'neginomial' is used to differentiate $n(x)$ with positive posynomial $p(x)$.

The broader definition of signomial functions enables the inclusion of expressions that previously could not be manipulated into GP-compatible forms. For example, the Taylor series expansions that had limited compatibility with GP are now fully SP compatible. The standard form of a Signomial Program is written as [5]:

$$\begin{aligned} \text{minimize} \quad & \frac{p_0(x)}{n_0(x)} \\ \text{subject to} \quad & s_i(x) = 0, \quad i = 1, \ldots, N \\ & s_j(x) \leq 0, \quad j = 1, \ldots, M \end{aligned} \tag{1.11}$$

This definition requires $n(x)$ to be nonzero, which is fulfilled by nature of the log-log transformation. The $n(x)$ terms can then be added and divided on both sides of the equation to make the expression constrained or equal to 1, thus becoming [18] [5]:

$$\begin{aligned} \text{minimize} \quad & \frac{p_0(x)}{n_0(x)} \\ \text{subject to} \quad & \frac{p_i(x)}{n_i(x)} = 1, \quad i = 1, \ldots, N \\ & \frac{p_j(x)}{n_j(x)} \leq 1, \quad j = 1, \ldots, M \end{aligned} \tag{1.12}$$

SP functions are not convex with a log-log transformation, and solutions are not guaranteed to

be globally optimal. Instead, SP problems become difference-of-convex optimization problems which have readily available solution algorithms [28, 29]. An algorithm of interest is Difference of Convex Algorithms (DCA), also referred to as the Convex-Concave Procedure. Essentially, the SP problem is broken down into a sequence of GP problems, enabled by Equation 1.12. The GP problems are solved until convergence, and the solution is taken as a local optimum [18]. Notably, this work uses a Penalty Convex-Concave Procedure (PCCP) [30] for a more relaxed and robust DCA method.

1.6 Python-based Model and Solver

The modeling and optimization for this work is done in Pyomo [31], a Python-based tool specially designed for modeling and optimization applications. Pyomo features simple modeling syntax and access to numerous solvers and libraries for flexibility in formulations and interfaces. Pyomo provides an effective means to define variables, constants, constraints, and objective functions while maintaining solver independence. Seamless integration is available for a wide range of solvers, including linear, quadratic, nonlinear, mixed-integer, stochastic, and bilevel programming [31]. Python libraries such as Matplotlib [32] and Pandas [33] are useful post-processing and data visualization tools.

In this work, Pyomo is the modeling interface for the SP-compatible Lifting-Line Theory formulation. The Engineering Design Interface (EDI) package built on Pyomo is used for solving SP compatible formulations. EDI uses PCCP to approximate signomial terms through a sequence of geometric programs. An internally developed Pyomo wrapper and the CVXOPT package [20] solve the GP using a primal-dual interior-point method. Numpy [34] is used as a syntax for array processing and results are visualized using Matplotlib [32] and Pandas [33].

1.7 Research Contributions

GP and SP aircraft models such as those by Kirschen [5], Hoburg and Abeel [6], and Brown et al. [9, 23] use fitted data or simplification methods for computing induced drag. As a

result, SP optimizations have historically been limited in their potential to explore design trade-offs between wing aerodynamics and key sizing characteristics such as span, aspect ratio, and general wing geometry.

This work begins to address the gap by developing an SP compatible model for induced drag using a geometrically flexible Lifting-Line Theory formulation. Formulating the relations to be SP-compatible enables aerodynamic performance optimization along with aircraft design sizing variables. The advantages of SP and GP optimization are maintained while providing a method for calculating wing aerodynamics within the same framework, avoiding the expense of high fidelity black-box tools.

CHAPTER 2

CLASSICAL LIFTING-LINE THEORY

One method of analyzing the force components on a body is Prandtl's classical Lifting-Line Theory (LLT) [35], which solves for the lift distribution and induced drag of a given wing in flight. While LLT is lower fidelity than advanced modern methods, it is often more practical for design. LLT represents the wing as some number of constant strength vortices, which have strength Γ_i. Once the Γ_i values are determined, aerodynamic characteristics such as lift, drag, and downwash can be calculated. In this work, the LLT model is formulated using equations from Drela's lifting-line analysis [36], which expands upon Prandtl's classical form [37].

2.1 Lifting-Line Theory Foundation

The vortex filament is introduced as an imaginary curve that induces flow circulation around its path, as shown in Figure 2.1.

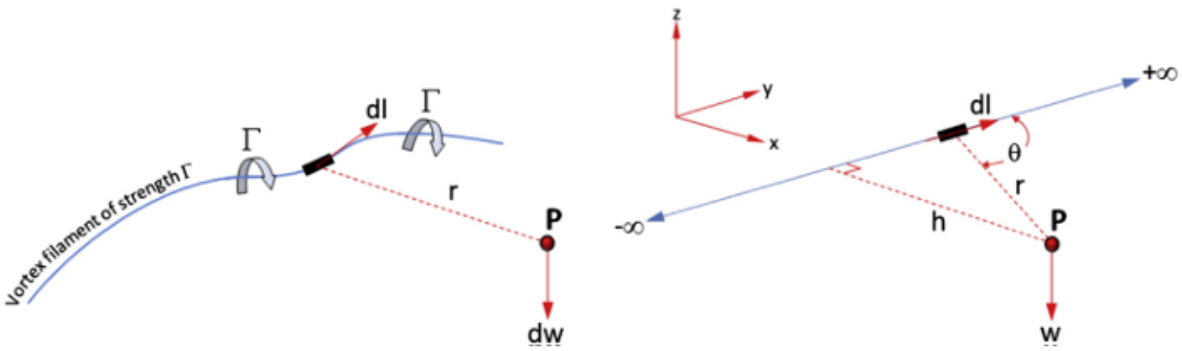

FIGURE 2.1. A curved vortex filament (left) and a straight, infinite vortex filament (right) [38].

The strength of the vortex filament circulation induction is referred to as Γ. By letting $d\ell$ be an infinitesimal segment of the filament and P some arbitrary point in space, the induced velocity dw at point P is found using the Biot–Savart law [37]:

$$dw = \frac{\Gamma}{4\pi} \frac{d\ell \times r}{|r|^3} \tag{2.1}$$

The Biot-Savart Law is applied to some number of straight and infinitely long vortex filaments. The previous equation is integrated along the total length of the filament to find the total induced velocity at the point P [37]:

$$w = \int_{-\infty}^{\infty} \frac{\Gamma}{4\pi} \frac{\mathrm{d}\boldsymbol{\ell} \times \mathbf{r}}{|\mathbf{r}|^3} = \int_{-\infty}^{\infty} \frac{\Gamma}{4\pi} \frac{\sin\theta}{|\mathbf{r}|^3} \tag{2.2}$$

Solving the integral results in the simplified induced velocity expression [37]:

$$w = \frac{\Gamma}{2\pi h} \tag{2.3}$$

Consider a similar vortex filament, which is bound by some point A and goes to infinity. This vortex filament is classified as semi-infinite, and is used to form the vortex system used in LLT. The integral from Equation 2.2 instead becomes:

$$w = \int_{A}^{\infty} \frac{\Gamma}{4\pi} \frac{\sin\theta}{|\mathbf{r}|^3} \tag{2.4}$$

and solving the integral results in the new induced velocity at point P [37]:

$$w = \frac{\Gamma}{4\pi h} \tag{2.5}$$

Helmholz's Vortex Theorems apply the concept of vortex filaments to inviscid, incompressible fluid flow. The theorems are listed for convenience [37]:

1. The strength of a vortex line is constant along its length.
2. A vortex line cannot terminate within a fluid; it must either extend to the boundaries of the fluid or form a closed path.
3. A fluid element that is initially irrotational remains irrotational.

Applying Helmholz's Theorems to a three-dimensional wing reveals a system consisting of three distinct vortex filaments. This system is known as a horseshoe vortex, shown in Figure 2.2. One

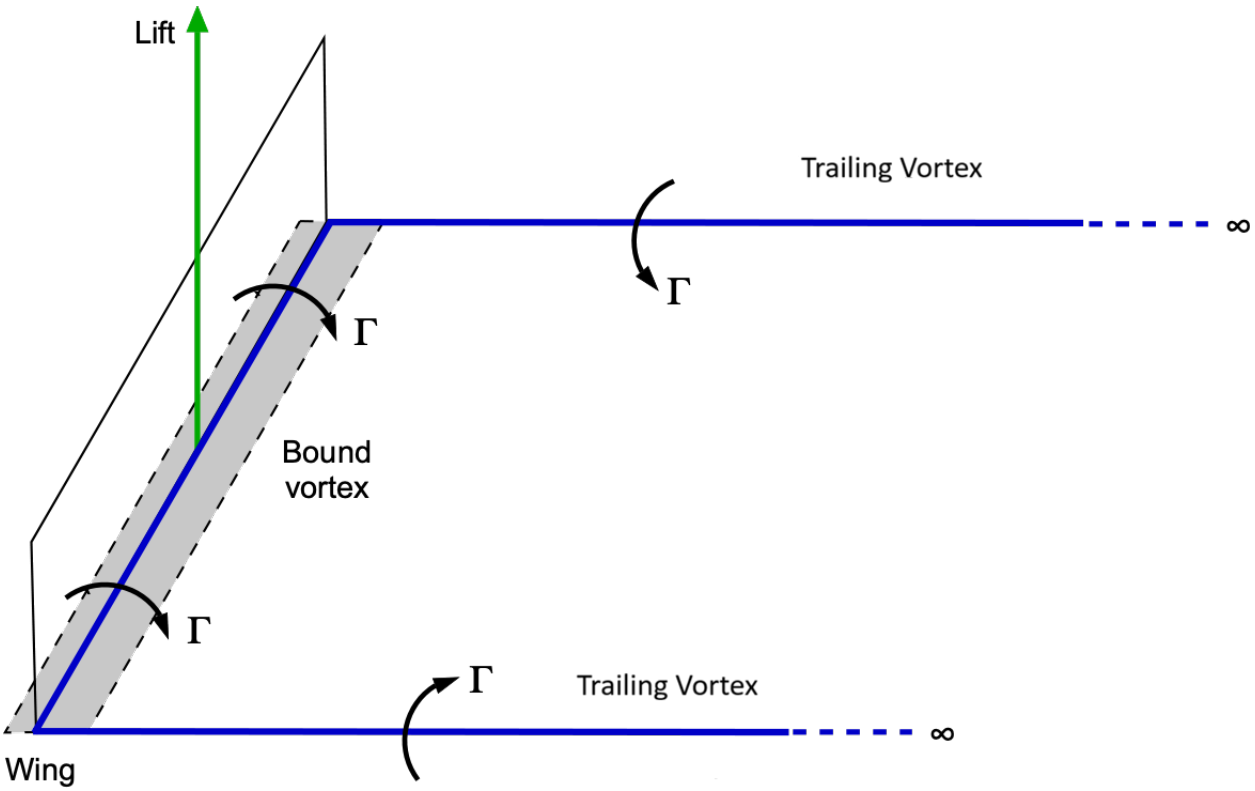

FIGURE 2.2. A horseshoe vortex on a finite wing [39].

filament reaches from infinity to the left wingtip, referred to as a trailing vortex. A second filament goes from the left wingtip to the right wingtip, representing a finite wing as a bound vortex. The third filament is another trailing vortex that reaches from the right wingtip to infinity. The three vortices have a constant strength Γ and induce a flow field in the system.

2.2 Prandtl's LLT Model

The initial horseshoe vortex is limited in fidelity when defining the entire wing. Prandtl addresses this by adding additional horseshoe vortices to the system, superimposed along the bound vortex. The line is called the lifting-line and is the namesake of Lifting-Line Theory. The

lifting-line reaches across the wingspan from $\frac{-b}{2}$ to $\frac{b}{2}$, as shown in Figure 2.3 [37]. The vortices are all constant strength, but by nature of the superposition, the vortices closer to the center of the wing typically have greater strength than those near the wingtip.

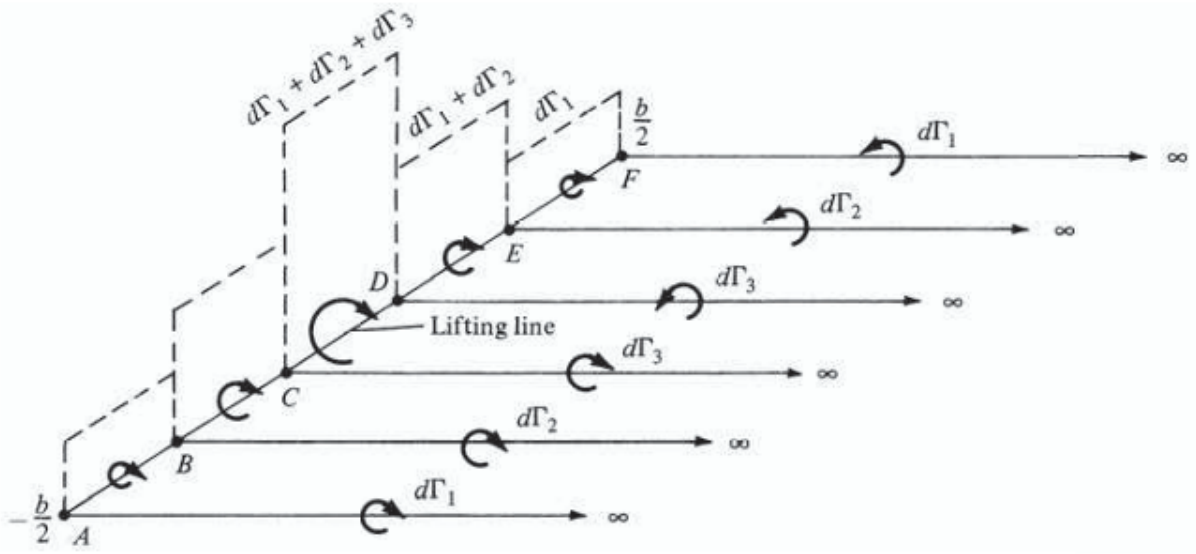

FIGURE 2.3. The lifting-line with a finite number of horseshoe vortices [37].

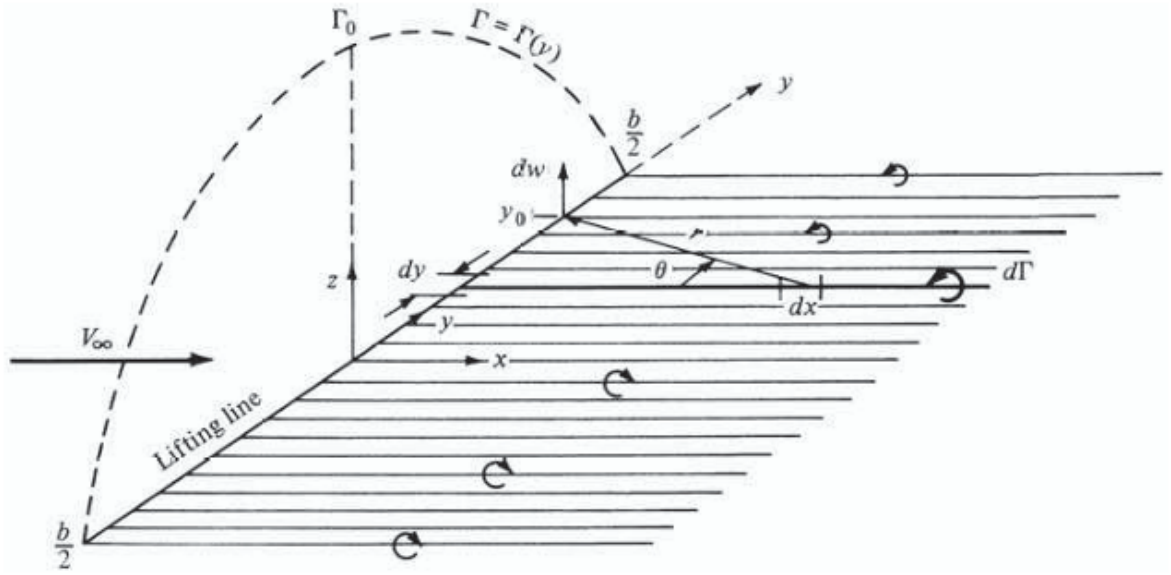

FIGURE 2.4. The lifting-line with an infinite number of horseshoe vortices, becoming a continuous vortex sheet [37].

A superposition of infinite horseshoe vortices results in a continuous vortex sheet, as shown in Figure 2.4. Consider one segment of the lifting-line, which is now represented by dy along the spanwise coordinate y. Applying the Biot-Savart Law to the sheet results in a total induced velocity of:

$$w(y_0) = -\frac{1}{4\pi}\int_{-b/2}^{b/2} \frac{\mathrm{d}\Gamma/\mathrm{d}y}{y_0 - y}\,\mathrm{d}y \tag{2.6}$$

where y_0 is some point along the lifting-line. The minus sign is used to represent the inverse relationship between dw and $d\Gamma$. Now consider the airfoil at y_0 as shown in Figure 2.5. Assuming the horizontal axis y and the vertical axis z, the angle of attack induced by the downwash is found with geometry:

$$\alpha_i(y_0) = \tan^{-1}\left(\frac{w(y_0)}{V_\infty}\right) \approx \frac{w(y_0)}{V_\infty} \tag{2.7}$$

in which the approximation is valid for small angles ($\sin\alpha_i \approx \alpha_i$). Note the effective angle of attack as shown in Figure 2.5:

$$\alpha_{\text{eff}} = \alpha - \alpha_i \tag{2.8}$$

and the lift coefficient for a localized airfoil section:

$$c_\ell = 2\pi[\alpha_{\text{eff}}(y_0) - \alpha_{L=0}] \tag{2.9}$$

where $\alpha_{L=0}$ is the angle of zero lift and $\alpha_{eff} = \alpha_{\text{eff}}(y_0)$. The thin airfoil theoretical lift slope of 2π is used. Additionally, consider the localized Kutta-Joukowski theorem for lift distribution:

$$L'(y_0) = \rho V_\infty \Gamma(y_0) = \frac{1}{2}\rho V_\infty^2\, c(y_0)\, c_\ell \tag{2.10}$$

While the full derivation from [37] is not shown, rewriting the two rightmost expressions in Equation 2.10 to be in terms of c_ℓ and inserting into Equation 2.9 can be used to find an expression for

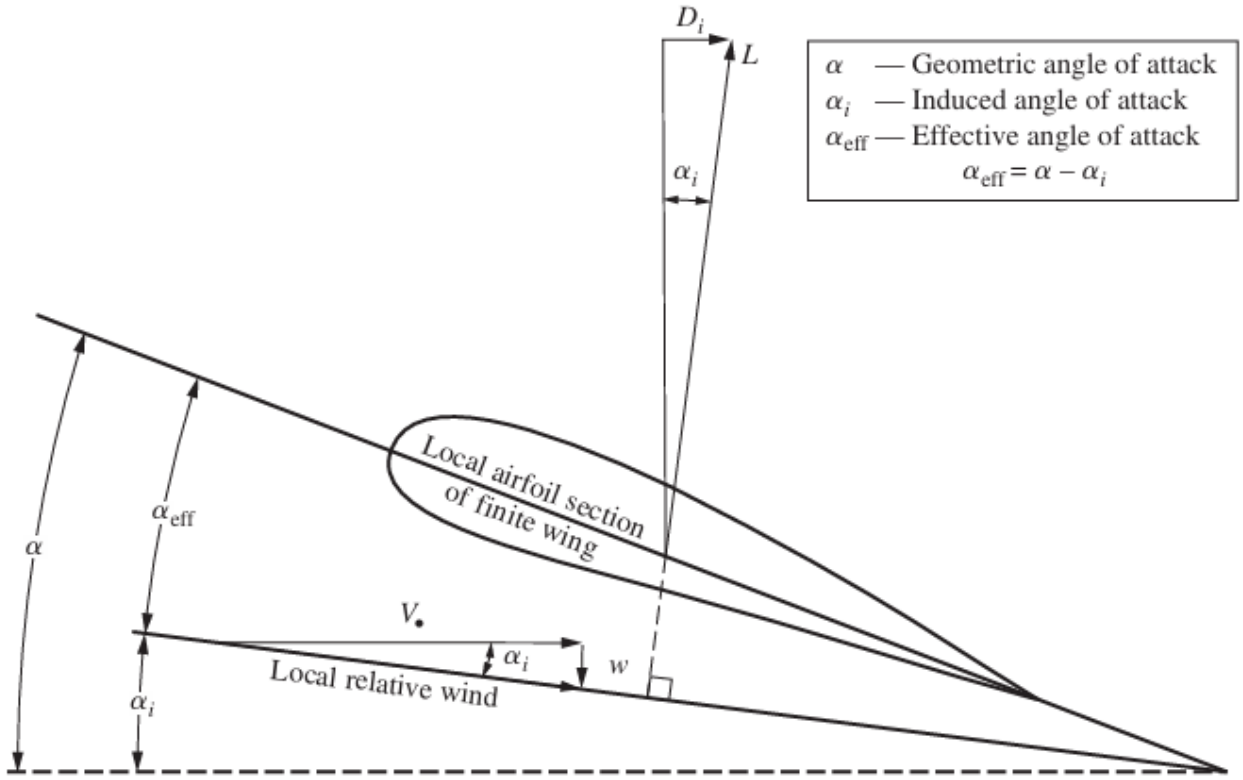

FIGURE 2.5. The angle of attack induced by downwash on the localized airfoil of the wing [37].

the angle of attack:

$$\alpha_{\text{eff}} = \alpha_{L=0}(y_0) + \frac{\Gamma(y_0)}{\pi V_\infty c(y_0)} \tag{2.11}$$

By putting all of the expressions in terms of the α_{eff} in Equation 2.8, the fundamental equation of Prandtl's lifting-line theory is found:

$$\alpha(y_0) = \alpha_{L=0}(y_0) + \frac{\Gamma(y_0)}{\pi V_\infty c(y_0)} + \frac{1}{4\pi V_\infty} \int_{-b/2}^{b/2} \frac{\mathrm{d}\Gamma(y)}{\mathrm{d}y} \frac{1}{y_0 - y} \mathrm{d}y \tag{2.12}$$

Solving the fundamental equation for a given finite wing at a given angle of attack and freestream velocity finds that $\Gamma = \Gamma(0)$ along the wingspan.

From the solution and Kutta-Joukowski Theorem, it is expounded that the lift distribution

is:

$$L'(y_0) = \rho V_\infty \Gamma(y_0) \tag{2.13}$$

$$L'(y_0) = \frac{1}{2}\rho V_\infty^2 \, c(y_0) \, c_\ell \tag{2.14}$$

The induced drag per unit span is:

$$D_i' = L_i' \sin \alpha_i = L_i' \alpha_i \tag{2.15}$$

using the same small angle approximation $\sin \alpha_i \approx \alpha_i$. Integrating over the span, the total lift is:

$$\int_{-b/2}^{b/2} \frac{\mathrm{d}\Gamma(y)}{\mathrm{d}y} \frac{1}{y_0 - y} \, \mathrm{d}y \tag{2.16}$$

and the total induced drag is:

$$D_i = \int_{-b/2}^{b/2} L'(y) \alpha_i(y) \, \mathrm{d}y \tag{2.17}$$

$$D_i = \rho_\infty V_\infty \int_{-b/2}^{b/2} \Gamma(y) \alpha_i(y) \, \mathrm{d}y \tag{2.18}$$

Using known wing characteristics, the induced drag coefficient can also be found:

$$C_{D_i} = \frac{D_i}{q_\infty S} = \frac{2}{V_\infty S} \int_{-b/2}^{b/2} \Gamma(y) \alpha_i \, \mathrm{d}y \tag{2.19}$$

With the exception of V_∞ and Γ, all terms in this equation are geometric properties of the wing. Thus, for a given freestream velocity V_∞, the circulation Γ can be determined. Once Γ is known, the lift, lift distribution, and induced drag can be calculated. Acquiring these values is the motivation for developing a Lifting-Line model.

2.3 Drela's Lifting-Line Model

Prandtl's model as framed in Section 2.2 is limited from assuming a high aspect ratio wing, in which the velocity of the vortex wake w_{wake} cannot be uniform. This prevents a local $\alpha_{\text{eff}}(y)$ from being found, so the equations for localized lift distribution (Equations 2.13-14) and induced drag per unit span (Equation 2.15) cannot be used [36]. While lifting-line theory is used for high aspect ratio aircraft, this work uses Drela's idealized expressions for general applicability to lifting bodies and various wake shapes. Drela uses broader analyses for idealized far-field force expressions. Prandtl's LLT equations are a basis, implemented into a discrete panel method to expand the model for general, non-planar wakes and additional constraint implementation to eventually set up the minimum induced drag optimization problem.

2.3.1 Total Drag

The total drag is divided into profile drag and induced drag components, which is done by breaking down the far-field drag. The far-field force expressions are simplified in terms of the trailing vortex wake characteristics, which allows for calculations directly relating aerodynamic forces and the flow-field. The Trefftz plane, located downstream of the lifting body, is introduced for trailing wake analysis, as shown in Figure 2.5 [36].

To determine the idealized far-field drag, the far-field drag integral (derived in detail in [36]) is reduced to the yz Trefftz plane.

$$D = \iint_{\text{TP}} \left[p_\infty - p - \rho\, u\, (u - V_\infty) \right] \, dy\, dz \tag{2.20}$$

In this expression p_∞ is the freestream static pressure, ρ is air density, p is local static pressure (excluding hydrostatic pressure), and u is local streamwise velocity. For flow-field idealization, wake roll-up is neglected, and the wake vortex sheet is assumed to be directly behind the trailing edge along the streamwise x-axis. The total velocity in the Trefftz plane is written as shown in

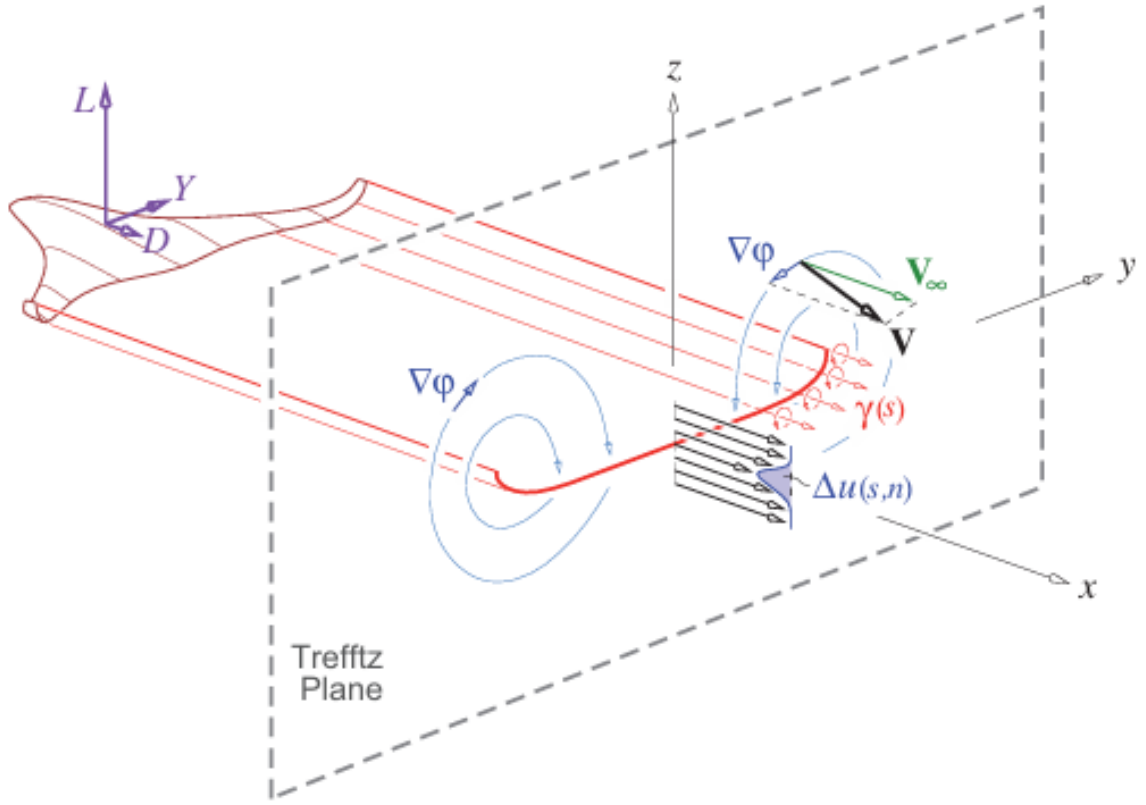

FIGURE 2.6. An idealized flow in the Trefftz plane behind a 3D lifting object, with trailing vortices and perturbation velocities [36].

Figure 2.6:

$$\mathbf{V} = (V_\infty + \Delta u)\ \hat{\mathbf{x}} + \nabla\varphi \tag{2.21}$$

in which $\nabla\varphi$ is introduced as the potential perturbation velocity linked to the streamwise vorticity, and shows the effect of each component on lift, sideforce, and induced drag. The term Δu is introduced as the streamwise viscous defect linked to the transverse vorticity.

Because Trefftz plane velocities are typically low, the incompressible Bernoulli equation can be used to calculate the pressure:

$$p = p_\infty + \frac{1}{2}\rho_\infty V_\infty^2 - \frac{1}{2}\rho_\infty |V_\infty\,\hat{\mathbf{x}} + \nabla\varphi|^2 \tag{2.22}$$

and after reducing becomes:

$$p = p_\infty - \rho_\infty V_\infty \varphi_x - \frac{1}{2}\rho_\infty \left(\varphi_y^2 + \varphi_z^2 + \varphi_x^2\right) \tag{2.23}$$

The velocity (Equations 2.21) and pressure (Equation 2.23) can be substituted into the far-field drag integral (Equation 2.20) to find the total drag, which is then broken down into profile drag and induced drag components [36].

$$D = D_p + D_i \tag{2.24}$$

$$D_p \equiv \iint \rho \left(V_\infty + 2\varphi_x + \Delta u\right)\left(-\Delta u\right)\, dS \tag{2.25}$$

$$D_i \equiv \iint \frac{1}{2}\rho_\infty \left(\varphi_y^2 + \varphi_z^2 - \varphi_x^2\right)\, dS \tag{2.26}$$

From Figure 2.5, $\nabla\varphi$ is assumed to be nearly parallel to the Trefftz plane, and φ_x is assumed to be negligible in both drag expressions. Thus, Equations 2.25 and 2.26 are simplified:

$$D_p \equiv \iint \rho \left(V_\infty + \Delta u\right)\left(-\Delta u\right)\, dS \tag{2.27}$$

$$D_i \equiv \iint \frac{1}{2}\rho_\infty \left(\varphi_y^2 + \varphi_z^2\right)\, dS \tag{2.28}$$

The idealized far field lift, fully derived in [36], is provided:

$$L = \rho_\infty V_\infty \int_{y_{\min}}^{y_{\max}} \Delta\varphi\, dy \tag{2.29}$$

and introduces the jump of the wake potential $\Delta\varphi$.

2.3.2 Discrete Panel Method

Before further defining the profile drag and induced drag components, the Discrete Panel Method is introduced as a method for integrating the forces over a general wake shape. The process is similar to applying a finite number of horseshoe vortices to the lifting-line, as shown in the

classical LLT, but allows for nonplanar wakes.

For SP compatibility, the formulation is done by substituting equivalent summations for integrals and geometric relations for trigonometric functions. Vectors are reduced to their components, which are separately used as constraints.

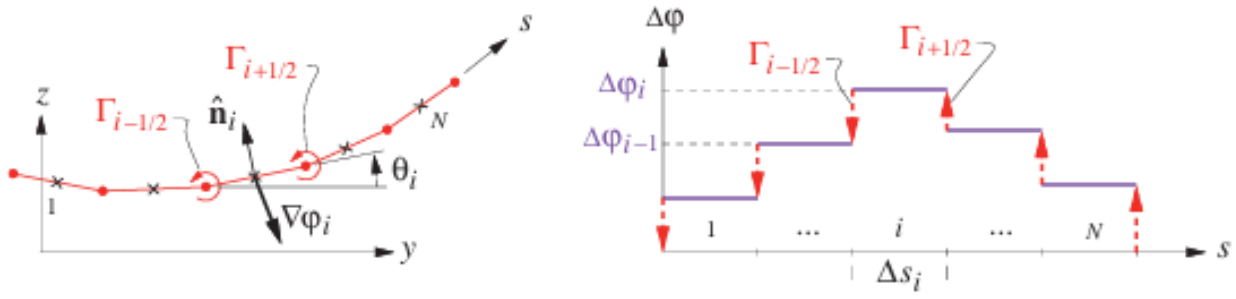

FIGURE 2.7. Wake panels in Trefftz plane, with panel angles, lengths, potentials, normal vectors, and circulations shown [36].

The wake configuration is discretized into some integer i = 1...N number of panels in the spanwise coordinate y and the vertical coordinate z, as shown in Figure 2.7. The panel endpoints are indexed as $y_{i+1/2}$ and $z_{i+1/2}$. For each panel the normal vector n is divided into components:

$$\boldsymbol{n}_i = \begin{cases} \frac{-1}{y_{i+1/2}-y_{i-1/2}}\hat{\boldsymbol{y}} + \frac{1}{z_{i+1/2}-z_{i-1/2}}\hat{\boldsymbol{z}} & \\ \hat{\boldsymbol{y}} & , y_{i-1/2} = y_{i+1/2} \\ \hat{\boldsymbol{z}} & , z_{i-1/2} = z_{i+1/2} \end{cases} \tag{2.30}$$

For nonplanar wake shapes, the angle from the horizontal of each panel θ_i is defined as:

$$\cos\theta_i = \frac{y_{i+1/2} - y_{i-1/2}}{\Delta s_i} \tag{2.31}$$

$$\sin\theta_i = \frac{z_{i+1/2} - z_{i-1/2}}{\Delta s_i} \tag{2.32}$$

where Δs_i is the ith panel length. From looking at the geometry in Figure 2.6, Δs_i is:

$$\Delta s_i = \sqrt{(y_{i+1/2} - y_{i-1/2})^2 + (z_{i+1/2} - z_{i-1/2})^2} \tag{2.33}$$

Using the definition of a unit vector for each panel:

$$\hat{\boldsymbol{n}}_i = \frac{\boldsymbol{n}_i}{|\boldsymbol{n}_i|} \tag{2.34}$$

and inserting Equations 2.30 and 2.33 results in the unit vector components:

$$\hat{n}_{iy} = -\frac{z_{i+1/2} - z_{i-1/2}}{\Delta s_i} \tag{2.35}$$

$$\hat{n}_{iz} = \frac{y_{i+1/2} - y_{\text{i-1/2}}}{\Delta s_i} \tag{2.36}$$

The y and z coordinates for the panel centers are simply:

$$y_i = \frac{y_{i+1/2} - y_{i-1/2}}{2} \tag{2.37}$$

$$z_i = \frac{z_{i+1/2} - z_{i-1/2}}{2} \tag{2.38}$$

Using the idealized far-field lift from Equation 2.29 and integrating over the wake panels results in the summation:

$$L = \rho_\infty V_\infty \sum_{i=1}^{N} \Delta\varphi_i \, \cos\theta \, \Delta s_i \tag{2.39}$$

The specified lift, which is later used as a constraint for potential jump $\Delta\varphi_i$, is found with Equation 2.39. Equation 2.31 is also inserted to eliminate $\cos\theta$, and conveniently reduces the expression to be solely in terms of the span, potential jump, and given freestream characteristics.

$$L_{spec} = \rho_\infty V_\infty \sum_{i=1}^{N} \Delta\varphi_i \, (y_{i+1/2} - y_{i-1/2}) \tag{2.40}$$

2.3.3 Induced Drag

The induced drag integral from Equation 2.28 is simplified using the gradient dot product identity and Gauss Theorem [36]:

$$D_i = \iint \frac{1}{2}\rho_\infty \nabla\varphi \cdot \nabla\varphi \; dS \tag{2.41}$$

$$= \iint \frac{1}{2}\rho_\infty \nabla \cdot (\varphi\nabla\varphi) \; dS \tag{2.42}$$

$$= \oint \frac{1}{2}\rho_\infty \varphi \nabla\varphi \cdot \hat{\mathbf{n}} \; dl \tag{2.43}$$

in which l is the arc length of the outer contour and $\hat{\mathbf{n}}$ points outward. Simplifying the surface integral to a singular integral along the wake and substituting $\delta\varphi$ results in:

$$D_i = -\frac{1}{2}\,\rho_\infty \int_0^{s_{\max}} \Delta\varphi \,\frac{\partial\varphi}{\partial n}\, ds \tag{2.44}$$

The lifting-line circulation Γ is defined as the potential difference between adjacent points. Since the boundary conditions are only adjacent to one point, they are defined using the same potential difference value:

$$\Gamma_{1/2} = -\Delta\varphi_1 \tag{2.45}$$

$$\Gamma_{N+1/2} = \Delta\varphi_N \tag{2.46}$$

The middle Γ values are the trailing vortex strength:

$$\Gamma_{i-1/2} = \Delta\varphi_{i-1} - \Delta\varphi_i, \; i = 2, ..., N \tag{2.47}$$

and positive about the x-axis. The gradient of the potential $\nabla\varphi_i$ is introduced as a 2D perturbation velocity field in the Trefftz plane. The expression uses a 2D superposition integral which takes

the velocity at each panel midpoint for the velocity, fully detailed in [36].

$$\nabla\varphi_i = \frac{1}{2\pi}\sum_{j=1}^{N+1}\Gamma_{j-\frac{1}{2}}\frac{\hat{\mathbf{x}}\times(\mathbf{r}_i-\mathbf{r}_{j-\frac{1}{2}})}{\left|\mathbf{r}_i-\mathbf{r}_{j-\frac{1}{2}}\right|^2} = \frac{1}{2\pi}\sum_{j=1}^{N+1}\Gamma_{j-1/2}\frac{-(z_i-z_{j-\frac{1}{2}})\hat{\mathbf{y}}+(y_i-y_{j-\frac{1}{2}})\hat{\mathbf{z}}}{(y_i-y_{j-1/2})^2+(z_i-z_{j-1/2})^2} \tag{2.48}$$

The numerator is split into components for visibility, and the fully expanded form separately includes the boundary conditions.

$$\begin{aligned}\nabla\varphi_i &= \frac{1}{2\pi}\sum_{j=1}^{N+1}\Gamma_{j-1/2}\left(\frac{-(z_i-z_{j-1/2})}{(y_i-y_{j-1/2})^2+(z_i-z_{j-1/2})^2}\hat{\boldsymbol{y}}+\frac{(y_i-y_{j-1/2})}{(y_i-y_{j-1/2})^2+(z_i-z_{j-1/2})^2}\hat{\boldsymbol{z}}\right)\\ &= \frac{1}{2\pi}\left[(-\Delta\varphi_1)\left(\frac{-(z_i-z_{1/2})}{(y_i-y_{1/2})^2+(z_i-z_{1/2})^2}\hat{\boldsymbol{y}}+\frac{(y_i-y_{1/2})}{(y_i-y_{1/2})^2+(z_i-z_{1/2})^2}\hat{\boldsymbol{z}}\right)\right.\\ &+\sum_{j=2}^{N}(\Delta\varphi_{j-1}-\Delta\varphi_j)\left(\frac{-(z_i-z_{j-1/2})}{(y_i-y_{j-1/2})^2+(z_i-z_{j-1/2})^2}\hat{\boldsymbol{y}}+\frac{(y_i-y_{j-1/2})}{(y_i-y_{j-1/2})^2+(z_i-z_{j-1/2})^2}\hat{\boldsymbol{z}}\right)\\ &\left.+\Delta\varphi_N\left(\frac{-(z_i-z_{N+1/2})}{(y_i-y_{N+1/2})^2+(z_i-z_{N+1/2})^2}\hat{\boldsymbol{y}}+\frac{(y_i-y_{N+1/2})}{(y_i-y_{N+1/2})^2+(z_i-z_{N+1/2})^2}\hat{\boldsymbol{z}}\right)\right]\end{aligned} \tag{2.49}$$

The normal component of the potential gradient is computed with a dot product:

$$\frac{\partial\varphi}{\partial n_i} = \nabla\varphi_i\cdot\hat{\boldsymbol{n}}_i = \nabla\varphi_{iy}\hat{n}_{iz}+\nabla\varphi_{iz}\hat{n}_{iz} \tag{2.50}$$

and after expanding the $\hat{\boldsymbol{n}}_i$ terms becomes:

$$\frac{\partial\varphi}{\partial n_i} = \nabla\varphi_{iy}\frac{(z_{i+1/2}-z_{i-1/2})}{\Delta s_i}-\nabla\varphi_{iz}\frac{(y_{i+1/2}-y_{\text{i-1/2}})}{\Delta s_i} \tag{2.51}$$

The $\nabla\varphi_{iy}$ and $\nabla\varphi_{iz}$ components are fully expanded, with the $\nabla\varphi_{iy}$ component written as:

$$\begin{aligned}
\nabla\varphi_{iy} &= \frac{1}{2\pi}\sum_{j=1}^{N+1}\Gamma_{j-1/2}\left(\frac{-(z_i - z_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right) \\
&= \frac{1}{2\pi}\left[(-\Delta\varphi_1)\left(\frac{-(z_i - z_{1/2})}{(y_i - y_{1/2})^2 + (z_i - z_{1/2})^2}\right)\right. \\
&+ \sum_{j=2}^{N}\left(\Delta\varphi_{j-1} - \Delta\varphi_j\right)\left(\frac{-(z_i - z_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right) \\
&+ \left.\Delta\varphi_N\left(\frac{-(z_i - z_{N+1/2})}{(y_i - y_{N+1/2})^2 + (z_i - z_{N+1/2})^2}\right)\right]
\end{aligned} \tag{2.52}$$

and the expanded $\nabla\varphi_{iz}$ as:

$$\begin{aligned}
\nabla\varphi_{iz} &= \frac{1}{2\pi}\sum_{j=1}^{N+1}\Gamma_{j-1/2}\left(\frac{y_i - y_{j-1/2}}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right) \\
&= \frac{1}{2\pi}\left[(-\Delta\varphi_1)\left(\frac{y_i - y_{1/2}}{(y_i - y_{1/2})^2 + (z_i - z_{1/2})^2}\right)\right. \\
&+ \sum_{j=2}^{N}\left(\Delta\varphi_{j-1} - \Delta\varphi_j\right)\left(\frac{y_i - y_{j-1/2}}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right) \\
&+ \left.\Delta\varphi_N\left(\frac{y_i - y_{N+1/2}}{(y_i - y_{N+1/2})^2 + (z_i - z_{N+1/2})^2}\right)\right]
\end{aligned} \tag{2.53}$$

The expanded forms of the expressions are listed for their use in the SP LLT formulation, discussed in Section 3.3. Equation 2.51 is condensed as an Aerodynamic Influence Coefficient (AIC) matrix A_{ij}. The potential jumps for a given wake geometry are used to find the normal velocities:

$$\frac{\partial\varphi}{\partial n_i} = \nabla\varphi_i \cdot \hat{\boldsymbol{n}}_i = \sum_{j=1}^{N} A_{ij}\Delta\varphi_j \tag{2.54}$$

Finally, the induced drag integral approximates Equation 2.44 with a summation over the panels.

$$D_i = -\frac{1}{2}\rho_\infty \sum_{i=1}^{N}\Delta\varphi_i \frac{\partial\varphi}{\partial n_i}\Delta s_i \tag{2.55}$$

2.3.4 Optimum Potential Jump

For induced drag minimization, the optimum $\frac{\partial \varphi}{\partial n_i}$ and corresponding optimum $\Delta\varphi_i$ are the desired unknown variables. However, Equation 2.54 cannot be solved with two unknown values and more relations must be introduced. The minimum induced drag uses a fixed specified lift, which requires that the lift variation is equal to zero, expressed as $\delta L = 0$. Putting the lift integral from Equation 2.29 in terms of the panel lengths gives:

$$L = \rho_\infty V_\infty \int \Delta\phi \cos\theta \, ds \tag{2.56}$$

and applying the fixed lift condition results in:

$$\delta L = \rho_\infty V_\infty \int \Delta\delta\varphi \, \cos\theta \, ds = 0 \tag{2.57}$$

By definition, the minimum induced drag is required to be stationary, expressed as $\delta D_i = 0$. Linearizing the D_i from Equation 2.55 gives δD_i as the expression:

$$\delta D_i = -\frac{1}{2}\rho_\infty \int \left(\Delta\delta\varphi \frac{\partial \varphi}{\partial n} + \Delta\varphi \frac{\partial \delta\varphi}{\partial n} \right) ds \tag{2.58}$$

and with some simplifications, detailed in [36], becomes:

$$\delta D_i = -\rho_\infty \int \Delta\delta\varphi \frac{\partial \varphi}{\partial n} \, ds \tag{2.59}$$

The D_i optimality condition $\delta D_i = 0$ is now imposed on each panel using the potential gradient along the normal components, resulting in a normal velocity distribution:

$$\frac{\partial \varphi}{\partial n}(s) = \Lambda \cos\theta(s) \tag{2.60}$$

with some introduced constant variable Λ. This $\frac{\partial \varphi}{\partial n}$ is proportional to the local $\cos\theta$, and thus the corresponding local panel geometry. The optimality condition is also satisfied, shown by first inserting Equation 2.60 into Equation 2.58:

$$\delta D_i = -\rho_\infty \int \Delta\delta\varphi \, \Lambda \cos\theta \, ds \tag{2.61}$$

Next, comparing Equation 2.61 to Equation 2.57 gives:

$$\delta D_i = -\frac{\Lambda}{V_\infty} \delta L = 0 \tag{2.62}$$

confirming the validity of Equation 2.60 [36]. Combining Equation 2.54 with Equation 2.60 yields:

$$\frac{\partial \varphi}{\partial n_i} = \nabla\varphi_i \cdot \hat{\boldsymbol{n}}_i = \sum_{j=1}^{N} A_{ij} \Delta\varphi_j = \Lambda \cos\theta_i \tag{2.63}$$

The terms on the right side of Equation 2.63 are expressed at each control point along the wake:

$$\sum_{j=1}^{N} A_{ij} \, \Delta\varphi_j \; - \; \Lambda \cos\theta_i \; = \; 0, \qquad (i = 1, \ldots, N) \tag{2.64}$$

Using Equation 2.31 expresses $\cos\theta_i$ in terms of the panel coordinates:

$$\sum_{j=1}^{N} A_{ij} \, \Delta\varphi_j \; - \; \Lambda \frac{y_{i+1/2} - y_{i-1/2}}{\Delta s_i} \; = \; 0, \qquad (i = 1, \ldots, N) \tag{2.65}$$

Equation 2.65 is used with the specified lift (Equation 2.41), listed again for convenience, to construct an $(N + 1) \times (N + 1)$ linear system to solve for $\Delta\varphi_i$ and Λ.

$$L_{spec} = \rho_\infty V_\infty \sum_{i=1}^{N} \Delta\varphi_i \, (y_{i+1/2} - y_{i-1/2})$$

The $\Delta\varphi_i$ and Λ values are then used with the AIC matrix to find $\frac{\partial\varphi}{\partial n_i}$, as shown in Equation 2.54. Finally, the minimum induced drag correlated with the optimum potential jumps can be calculated by using Equation 2.55.

With the lifting-line equations established, the next step is assembling them into a formulation for an SP compatible model. The rigorous mathematics from Chapter 2 are combined with the SP compatible definition introduced in Chapter 1 to build an LLT analysis model for optimization. The following section demonstrates LLT modeling for the minimum induced drag optimization problem.

CHAPTER 3

LIFTING-LINE PROBLEM FORMULATION

3.1 Minimum Induced Drag

In this work, the lifting-line equations are modeled for an induced drag optimization problem on a general non-planar wake. The design problem is finding the minimum induced drag given a fixed lift L_{spec}, some wake shape in y and z coordinates, and freestream characteristics ρ_∞ and V_∞. The given values are assumed constant. The desired variables are $\Delta\varphi_i$ and $\frac{\partial\varphi}{\partial n_i}$, which combine to demonstrate the lift distribution for a wake shape and are used to calculate D_i. To compare and validate the results for the SP formulation, the minimum induced drag problem is solved with both the Drela equation and novel SP compatible methodologies. In this chapter, both methodologies are written as implemented into the Pyomo modeling framework. The Drela formulation is listed first, followed by the SP formulation along with detailed changes between the methods. The full Python codes that describe each model are listed in Appendix A.

3.2 Lifting-Line Theory Drela Formulation

The Drela formulation seeks to optimize for the objective:

$$\text{minimize } D_i$$

Panel coordinates and geometry are defined by:

$$y_i = \frac{y_{i+1/2} - y_{i-1/2}}{2} \tag{3.1}$$

$$z_i = \frac{z_{i+1/2} - z_{i-1/2}}{2} \tag{3.2}$$

$$\cos\theta_i = \frac{y_{i+1/2} - y_{i-1/2}}{\Delta s_i} \tag{3.3}$$

$$\Delta s_i = \sqrt{(y_{i+1/2} - y_{i-1/2})^2 + (z_{i+1/2} - z_{i-1/2})^2} \tag{3.4}$$

Because cos and sin are not used in the SP version, for consistency the Δs coordinates are manipulated using trigonometric definitions to be in terms of the y and z coordinates instead of θ. The normal unit vector components are defined as:

$$\hat{n}_{iy} = -\frac{z_{i+1/2} - z_{i-1/2}}{\Delta s_i} \tag{3.5}$$

$$\hat{n}_{iz} = \frac{y_{i+1/2} - y_{\text{i-1/2}}}{\Delta s_i} \tag{3.6}$$

The $\hat{n}_i$ vectors are not explicitly defined with equations in the Drela text but are simply derived with geometric definitions based on the middle of each panel. The circulation and potential jumps are defined as:

$$\Gamma_{1/2} = -\Delta\varphi_1, \tag{3.7}$$

$$\Gamma_{i-1/2} = \Delta\varphi_{i-1} - \Delta\varphi_i, i = 2...N \tag{3.8}$$

$$\Gamma_{N+1/2} = \Delta\varphi_N \tag{3.9}$$

The Γ values are directly substituted as $\Delta\varphi_i$ values for simplification in the code. This approximation is valid with high aspect ratio wings, which are the only configurations considered for the scope of this work. Potential gradients are defined as:

$$\nabla\varphi_i = \frac{1}{2\pi}\sum_{j=1}^{N+1}\Gamma_{j-1/2}\frac{-(z_i - z_{j-\frac{1}{2}})\hat{\mathbf{y}} + (y_i - y_{j-\frac{1}{2}})\hat{\mathbf{z}}}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2} \tag{3.10}$$

The potential gradients are used in the AIC matrix and do not need to be separated into y and z components. The AIC matrix is formulated using the potential gradient and normals:

$$\nabla\varphi_i \cdot \hat{\boldsymbol{n}}_i = \sum_{j=1}^{N} A_{ij}\Delta\varphi_j \tag{3.11}$$

Note that $\Delta\varphi_j$ is not solved using this initial construction, but by the $(N + 1) \times (N + 1)$ linear

system which is defined as:

$$\sum_{j=1}^{N} A_{ij}\,\Delta\varphi_j \;-\; \Lambda\frac{y_{i+1/2}-y_{i-1/2}}{\Delta s_i} \;=\; 0, \qquad (i=1,\ldots,N) \tag{3.12}$$

$$L_{spec} = \rho_\infty V_\infty \sum_{i=1}^{N} \Delta\varphi_i \left(y_{i+1/2}-y_{i-1/2}\right) \tag{3.13}$$

The AIC matrix expression and specified lift linear system is solved to find $\Delta\varphi_i$. The potential gradient normal component can now be found:

$$\frac{\partial\varphi}{\partial n_i} = \nabla\varphi_i \cdot \hat{\boldsymbol{n}}_i = \sum_{j=1}^{N} A_{ij}\Delta\varphi_j \tag{3.14}$$

Finally, induced drag can be computed:

$$D_i = -\frac{1}{2}\rho_\infty \sum_{i=1}^{N} \Delta\varphi_i \frac{\partial\varphi}{\partial n_i}\Delta s_i \tag{3.15}$$

3.3 Signomial Programming Compatible Lifting-Line Theory Formulation

The Signomial Programming methodology largely remains the same as the Drela methodology in terms of the equations used. However, several variables are no longer included in the formulation due to SP incompatibility, and are directly substituted using placeholders.

The major difference between the methodologies is that the AIC matrix is no longer being used. The variables $\Delta\varphi$ and $\frac{\partial\varphi}{\partial n_i}$ are solved for with an SP compatible optimization outputs, and are values found through the SP solver instead of solving the linear system from the Drela equations. The equations and constraints used in the SP formulation are listed, with specific differences from the previous formulation listed and explained.

The objective function is the induced drag:

$$\text{minimize } D_i$$

The panel coordinates and geometry are the same as the Drela formulation:

$$y_i = \frac{y_{i+1/2} - y_{i-1/2}}{2} \tag{3.16}$$

$$z_i = \frac{z_{i+1/2} - z_{i-1/2}}{2} \tag{3.17}$$

$$\cos\theta_i = \frac{y_{i+1/2} - y_{i-1/2}}{\Delta s_i} \tag{3.18}$$

$$\sin\theta_i = \frac{z_{i+1/2} - z_{i-1/2}}{\Delta s_i} \tag{3.19}$$

$$\Delta s_i = \sqrt{(y_{i+1/2} - y_{i-1/2})^2 + (z_{i+1/2} - z_{i-1/2})^2} \tag{3.20}$$

The normal unit vectors are found with the geometry:

$$\hat{n}_{iy} = -\frac{z_{i+1/2} - z_{i-1/2}}{\Delta s_i} \tag{3.21}$$

$$\hat{n}_{iz} = \frac{y_{i+1/2} - y_{\text{i-1/2}}}{\Delta s_i} \tag{3.22}$$

The specified lift is provided, with $\cos\theta_i$ substituted for SP compatibility:

$$L_i' = \rho_\infty V_\infty \Delta\varphi_i (y_{i+1/2} - y_{i-1/2}) \tag{3.23}$$

Potential gradients are now expanded:

$$\begin{aligned}\nabla\varphi_{iy} &= \frac{1}{2\pi}\sum_{j=1}^{N+1}\Gamma_{j-1/2}\left(\frac{-(z_i - z_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right) \\ &= \frac{1}{2\pi}\left[(-\Delta\varphi_1)\left(\frac{-(z_i - z_{1/2})}{(y_i - y_{1/2})^2 + (z_i - z_{1/2})^2}\right)\right. \\ &+ \sum_{j=2}^{N}\left(\Delta\varphi_{j-1} - \Delta\varphi_j\right)\left(\frac{-(z_i - z_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right) \\ &+ \left.\Delta\varphi_N\left(\frac{-(z_i - z_{N+1/2})}{(y_i - y_{N+1/2})^2 + (z_i - z_{N+1/2})^2}\right)\right]\end{aligned} \tag{3.24}$$

$$\begin{aligned}\nabla\varphi_{iz} &= \frac{1}{2\pi}\sum_{j=1}^{N+1}\Gamma_{j-1/2}\left(\frac{(y_i - y_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right) \\ &= \frac{1}{2\pi}\left[(-\Delta\varphi_1)\left(\frac{(y_i - y_{1/2})}{(y_i - y_{1/2})^2 + (z_i - z_{1/2})^2}\right)\right. \\ &+ \sum_{j=2}^{N}\left(\Delta\varphi_{j-1} - \Delta\varphi_j\right)\left(\frac{(y_i - y_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right) \\ &+ \left.\Delta\varphi_N\left(\frac{(y_i - y_{N+1/2})}{(y_i - y_{N+1/2})^2 + (z_i - z_{N+1/2})^2}\right)\right]\end{aligned} \tag{3.25}$$

Depending on the location and wake geometry, each term in the $\nabla\varphi_i$ expressions can be positive or negative. Although SP allows for sums and differences in the constraints, the signs of each term must be the same for each iteration of the summation. To avoid being incompatible with SP, the equivalent expression is directly inserted. Using the potential gradients, the normal component is found:

$$\frac{\partial\varphi}{\partial n_i} \geq -\nabla\varphi_i \cdot \hat{n}_i = \nabla\varphi_{iy}\frac{(z_{i+1/2} - z_{i-1/2})}{\Delta s_i} - \nabla\varphi_{iz}\frac{(y_{i+1/2} - y_{\text{i-1/2}})}{\Delta s_i} \tag{3.26}$$

The $\frac{\partial\varphi}{\partial n_i}$ term is now a variable that is optimized via SP compatible solving methods and expressed as an inequality. A negative sign is also introduced in the code for SP compatibility. This change is addressed with another negative sign in the final induced drag expression to keep consistency

in the results. The induced drag is now expressed as:

$$D_i \geq \frac{1}{2}\rho_\infty \sum_{i=1}^{N} \Delta\varphi_i \frac{\partial\varphi}{\partial n_i} \Delta s_i \tag{3.27}$$

where coefficient $\frac{1}{2}\rho_\infty$ is initially negative but becomes positive to counteract the negative sign introduced in Equation 3.26. Additionally, as a desired optimization variable, D_i is now expressed as an inequality.

3.3.1 Formulation Comparison

A broad overview of the completed formulations is listed in Table 3.1. The equations are colorized in the table based on their characteristics within their respective problems. Constraints that are constants only depend on the given initial conditions from the problem statement. For the LLT equations, these are the wake panel coordinates, corresponding geometric definitions, the freestream velocity and density, and the specified lift.

Analysis Variables are defined as variables that are not initially given, and are directly solved using equations. The Drela formulation solves for the minimum induced drag through equations and solving a linear system. Thus, the induced drag and any associated variables that are solved and used to calculate the induced drag are Analysis Variables. Optimization Variables are defined as variables that are solved for by optimization methods, and are not directly solved with equations. Eliminated Variables are incompatible with SP and are not declared as variables in the formulation. Instead, they are directly substituted into the code.

The SP optimization model in this work only solves for Optimization Variables, and Analysis Variables from the Drela equations either become Optimization Variables or Eliminated Variables depending on their feasibility to be manipulated into SP compatible forms. Once identified, constraints are set as equality or inequality constraints based on the optimization objective. The objective D_i is an Analysis Variable in the Drela text and becomes an Optimization Variable for the SP model. The correlating $\frac{\partial\varphi}{\partial n_i}$ and $\Delta\varphi_i$ are not directly solved, and instead optimized to

minimize the induced drag. For the novel LLT model, despite being potentially SP compatible θ is eliminated, as direct substitution is simpler to implement than a Taylor series representation. The Γ, $\nabla\varphi_{iy}$, and $\nabla\varphi_{iz}$ values that can potentially change signs are eliminated and directly substituted. After the variables from the Drela formulation are all made into SP compatible Optimization Variables or eliminated, the LLT model is constructed.

	Drela Text Equations	SP-Compatible Model
Objective	D_i	D_i
Constraint 1	$y_i = \frac{y_{i+1/2}-y_{i-1/2}}{2}$	$y_i = \frac{y_{i+1/2}-y_{i-1/2}}{2}$
Constraint 2	$z_i = \frac{z_{i+1/2}-z_{i-1/2}}{2}$	$z_i = \frac{z_{i+1/2}-z_{i-1/2}}{2}$
Constraint 3	$\cos\theta_i = \frac{y_{i+1/2}-y_{i-1/2}}{\Delta s_i}$, $\sin\theta_i = \frac{z_{i+1/2}-z_{i-1/2}}{\Delta s_i}$	$\Delta s_i = \sqrt{(y_{i+1/2}-y_{i-1/2})^2 + (z_{i+1/2}-z_{i-1/2})^2}$
Constraint 4	$\hat{n}_{iy} = -\frac{z_{i+1/2}-z_{i-1/2}}{\Delta s_i}$	$\hat{n}_{iy} = -\frac{z_{i+1/2}-z_{i-1/2}}{\Delta s_i}$
Constraint 5	$\hat{n}_{iz} = \frac{y_{i+1/2}-y_{i-1/2}}{\Delta s_i}$	$\hat{n}_{iz} = \frac{y_{i+1/2}-y_{i-1/2}}{\Delta s_i}$
Constraint 6	$L_{spec} = \rho_\infty V_\infty \sum_{i=1}^{N} \Delta\varphi_i \cos\theta_i \Delta s_i$	$L_{spec} = \rho_\infty V_\infty \sum_{i=1}^{N} \Delta\varphi_i (y_{i+1/2} - y_{i-1/2})$
Constraint 7	$\Gamma_{1/2} = -\Delta\varphi_1$, $\Gamma_{i-1/2} = \Delta\varphi_{i-1} - \Delta\varphi_i,\ i = 2...N$, $\Gamma_{N+1/2} = \Delta\varphi_N$	$\Gamma_{1/2} = -\Delta\varphi_1$, $\Gamma_{i-1/2} = \Delta\varphi_{i-1} - \Delta\varphi_i,\ i = 2...N$, $\Gamma_{N+1/2} = \Delta\varphi_N$
Constraint 8	$\nabla\varphi_i = \frac{1}{2\pi}\sum_{j=1}^{N+1} \Gamma_{j-1/2} \frac{-(z_i - z_{j-1/2})\hat{\boldsymbol{y}} + (y_i - y_{j-1/2})\hat{\boldsymbol{z}}}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}$	$\nabla\varphi_i = \frac{1}{2\pi}\left[(-\Delta\varphi_1)\frac{-(z_i - z_{1/2})\hat{\boldsymbol{y}} + (y_i - y_{1/2})\hat{\boldsymbol{z}}}{(y_i - y_{1/2})^2 + (z_i - z_{1/2})^2}\right.$ $+ \sum_{j=2}^{N}(\Delta\varphi_{j-1} - \Delta\varphi_j)\frac{-(z_i - z_{j-1/2})\hat{\boldsymbol{y}} + (y_i - y_{j-1/2})\hat{\boldsymbol{z}}}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}$ $\left. + \Delta\varphi_N \frac{-(z_i - z_{N+1/2})\hat{\boldsymbol{y}} + (y_i - y_{N+1/2})\hat{\boldsymbol{z}}}{(y_i - y_{N+1/2})^2 + (z_i - z_{N+1/2})^2}\right]$
Constraint 9	$\nabla\varphi_{iy} = \frac{1}{2\pi}\sum_{j=1}^{N+1} \Gamma_{j-1/2}\left(\frac{-(z_i - z_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right)$	$\nabla\varphi_{iy} = \frac{1}{2\pi}\left[(-\Delta\varphi_1)\left(\frac{-(z_i - z_{1/2})}{(y_i - y_{1/2})^2 + (z_i - z_{1/2})^2}\right)\right.$ $+ \sum_{j=2}^{N}(\Delta\varphi_{j-1} - \Delta\varphi_j)\left(\frac{-(z_i - z_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right)$ $\left. + \Delta\varphi_N\left(\frac{-(z_i - z_{N+1/2})}{(y_i - y_{N+1/2})^2 + (z_i - z_{N+1/2})^2}\right)\right]$
Constraint 10	$\nabla\varphi_{iz} = \frac{1}{2\pi}\sum_{j=1}^{N+1} \Gamma_{j-1/2}\left(\frac{(y_i - y_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right)$	$\nabla\varphi_{iz} = \frac{1}{2\pi}\left[(-\Delta\varphi_1)\left(\frac{(y_i - y_{1/2})}{(y_i - y_{1/2})^2 + (z_i - z_{1/2})^2}\right)\right.$ $+ \sum_{j=2}^{N}(\Delta\varphi_{j-1} - \Delta\varphi_j)\left(\frac{(y_i - y_{j-1/2})}{(y_i - y_{j-1/2})^2 + (z_i - z_{j-1/2})^2}\right)$ $\left. + \Delta\varphi_N\left(\frac{(y_i - y_{N+1/2})}{(y_i - y_{N+1/2})^2 + (z_i - z_{N+1/2})^2}\right)\right]$
Constraint 11	$\frac{\partial\varphi}{\partial n_i} = \nabla\varphi_i \cdot \hat{\boldsymbol{n}}_i = \sum_{j=1}^{N} A_{ij}\Delta\varphi_j$ $= \nabla\varphi_{iy}\hat{n}_{iy} + \nabla\varphi_{iz}\hat{n}_{iz}$	$\frac{\partial\varphi}{\partial n_i} = -\nabla\varphi_i \cdot \hat{\boldsymbol{n}}_i$ $= -(\nabla\varphi_{iy}\hat{n}_{iy} + \nabla\varphi_{iz}\hat{n}_{iz})$
Constraint 12	$D_i = -\frac{1}{2}\rho_\infty \sum_{i=1}^{N} \Delta\varphi_i \frac{\partial\varphi}{\partial n_i}\Delta s_i$	$D_i = \frac{1}{2}\rho_\infty \sum_{i=1}^{N} \Delta\varphi_i \frac{\partial\varphi}{\partial n_i}\Delta s_i$
Analysis Variables	$D_i, \Delta\varphi_i, \Gamma_i, \frac{\partial\varphi}{\partial n_i}, \nabla\varphi_{iy}, \nabla\varphi_{iz}$	N/A
Optimization Variables	N/A	$D_i, \Delta\varphi_i, \frac{\partial\varphi}{\partial n_i}$
Eliminated Variables	N/A	$\theta_i, \Gamma_i, \nabla\varphi_{iy}, \nabla\varphi_{iz}$
Constants	$y_i, z_i, \Delta s_i, \theta_i, \hat{n}_{iy}, \hat{n}_{iz}, L_{spec}, \rho_\infty, V_\infty$	$y_i, z_i, \Delta s_i, \hat{n}_{iy}, \hat{n}_{iz}, L_{spec}, \rho_\infty, V_\infty$

TABLE 3.1. Overview of objectives, constraints, variables, and constants used in the lifting-line analysis models

CHAPTER 4

LLT OPTIMIZATION ON VARIOUS WAKE SHAPES

4.1 Test Problem and Motivation

After establishing the Lifting-Line methodologies and formulating the Signomial Programming-compatible model, test problems are required to validate the SP model against the known solution method from Drela. Multiple test cases are constructed using the minimum induced drag problem covered in Section 3.1. Constants and design variables are listed in Tables 4.1 and 4.2 respectively. The constant specified lift, freestream velocity and wingspan are selected based on specifications from the Cessna 172S [40]. While not explicitly defined as a design variable, the span is set at a constant value of 10.

The minimum induced drag problem is solved twice with two separate Python codes. The first code uses the Drela method and solves the AIC matrix, as detailed in Section 3.2. The second uses the SP formulation and solver to find the optimal variables and induced drag, as shown in Section 3.3. The expected results from the proven Drela methodology are compared with the values found using Signomial Programming to validate the SP LLT model.

TABLE 4.1. Constant Parameters and Reference Values used in LLT Model

Constant	Value	Units	Description
L_{spec}	10000	N	Specified Lift
ρ_∞	1.23	$\frac{kg}{m^3}$	Freestream Air Density
V_∞	50	$\frac{m}{s}$	Freestream Velocity
y		m	Spanwise Coordinate
z		m	Vertical Coordinate

4.1.1 Wake Panel Configurations

The codes are run using a variety of wake panel configurations: flat, parabolic, linear dihedral, sine, and asymmetric. The configurations are built with simple geometric functions using z as a function of y, shown in Figures 4.1-4.5. The flat wake shape is chosen as a simple initial test case, shown in Figure 4.1. Compared to the other geometries, the flat wake has the lowest in-

TABLE 4.2. Design Variables used in LLT Model

Variable	Units	Description
D_i	N	Induced Drag
$\Delta\varphi$	$\frac{m^2}{s}$	Potential Difference
Δs	m	Length along Wake
Γ	$\frac{m^2}{s}$	Lifting-Line Circulation
$\nabla\varphi_{i_y}$	$\frac{m}{s}$	Potential Gradient y-component
$\nabla\varphi_{i_z}$	$\frac{m}{s}$	Potential Gradient z-component
$\frac{\partial\varphi}{\partial n_i}$	$\frac{m}{s}$	Potential Derivative w.r.t. Unit Normal

duced drag, and without a span or loading constraint would theoretically extend infinitely. The resulting optimum lift distribution for this shape is elliptical, as indicated in the original Lifting-Line Theory [35]. The parabolic configuration, shown in Figure 4.2, and linear configuration, shown in Figure 4.3, are used to demonstrate the model on dihedral shapes that are slightly more complicated than the flat wake. The sine configuration, shown in Figure 4.4, is used to demonstrate the model on a complex curved configuration. The shape is slightly concave, resembling the wings of a bird. The configuration is reflected around the center point, using positive z values to keep sign consistency. Because the flat, parabolic, linear, and sine plots are symmetric, an asymmetric configuration was created for comparison. The asymmetric case is flat for the left half-span and parabolic for the right half-span, as shown in Figure 4.5. This creates a gradual dihedral curve, resembling a larger half-span.

To ensure that the y and z values are all positive, the wakes are shifted so that the initial minimum value of y and the minimum possible value of z are both 1. While not necessary for the constant wake geometries in this work, the shift is necessary for SP compatibility when considering the variable wake case, detailed further in Section 5.1.2. The plots are shown using using 20 panels as initial examples for control point and panel visualization. Potential jumps at each panel are plotted to represent the lift distribution.

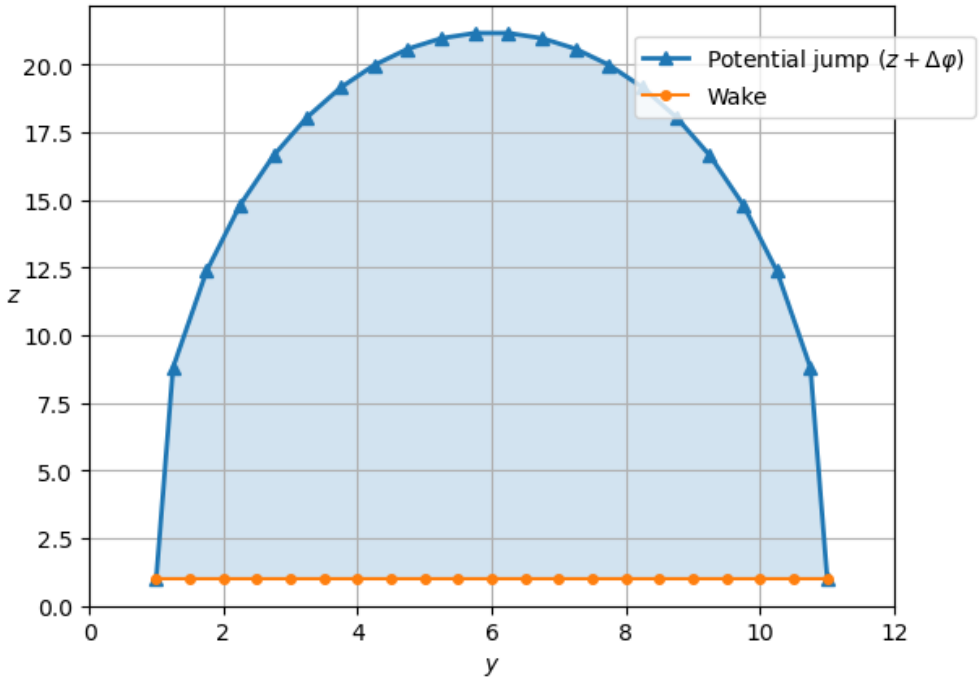

FIGURE 4.1. Lift Distribution over a flat panel configuration.

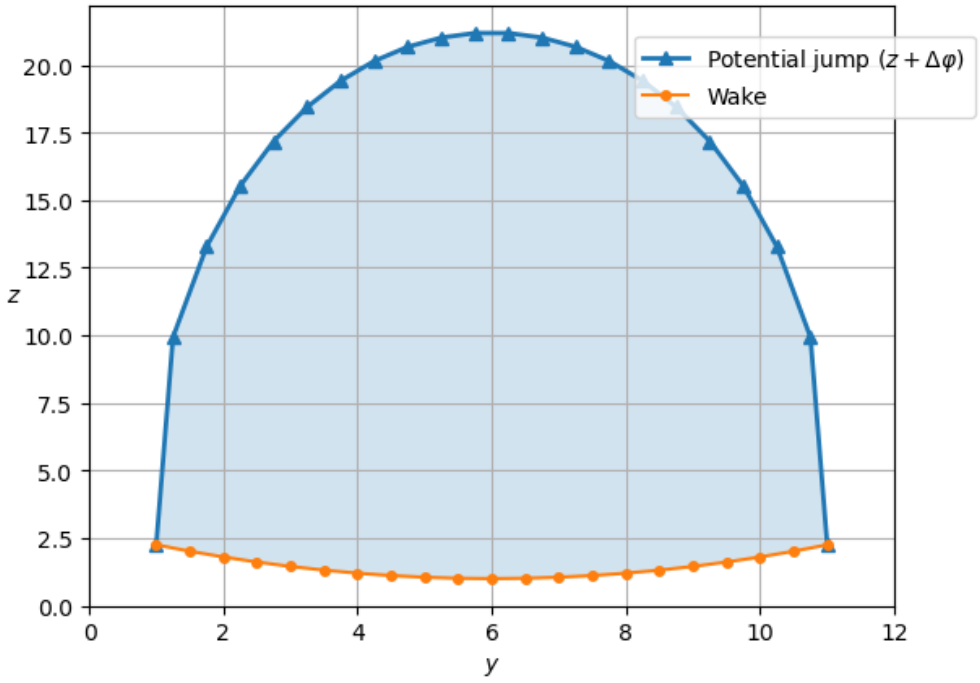

FIGURE 4.2. Lift Distribution over a parabolic panel configuration.

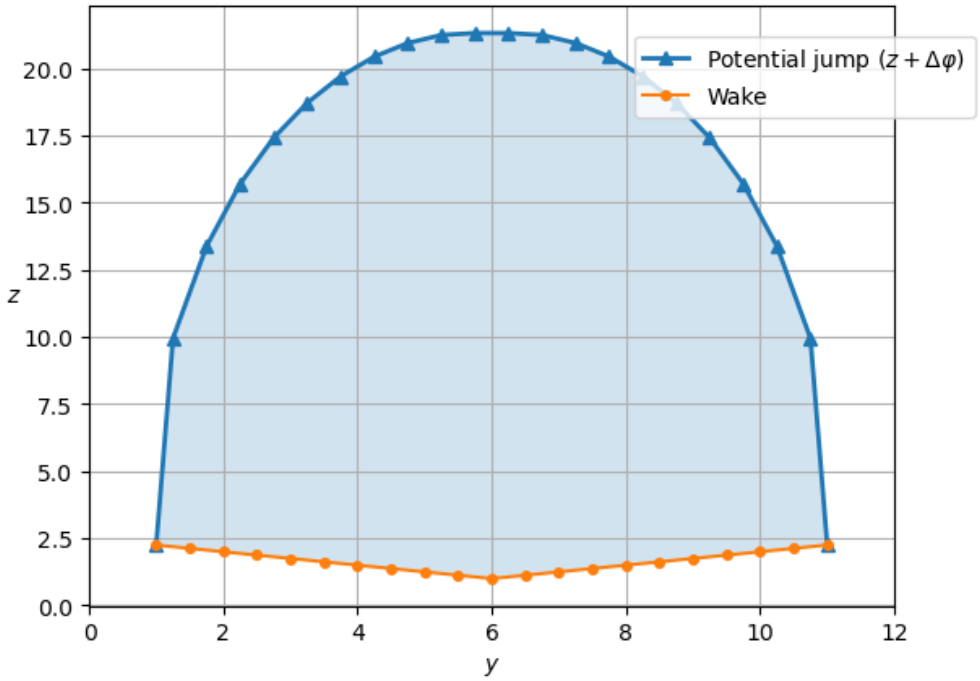

FIGURE 4.3. Lift Distribution over a linear dihedral panel configuration.

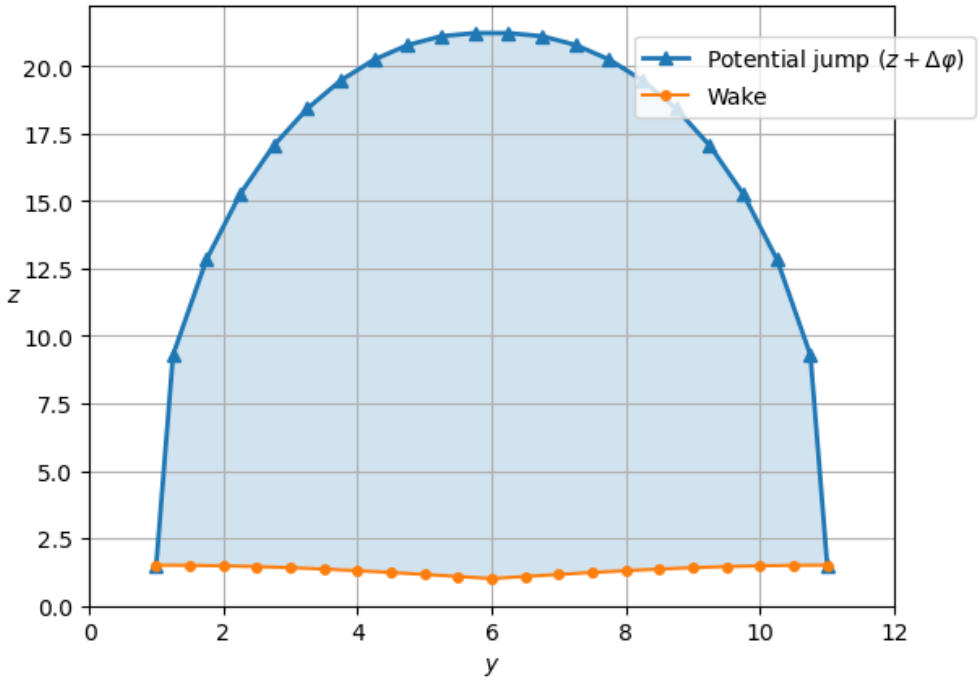

FIGURE 4.4. Lift Distribution over a sine panel configuration.

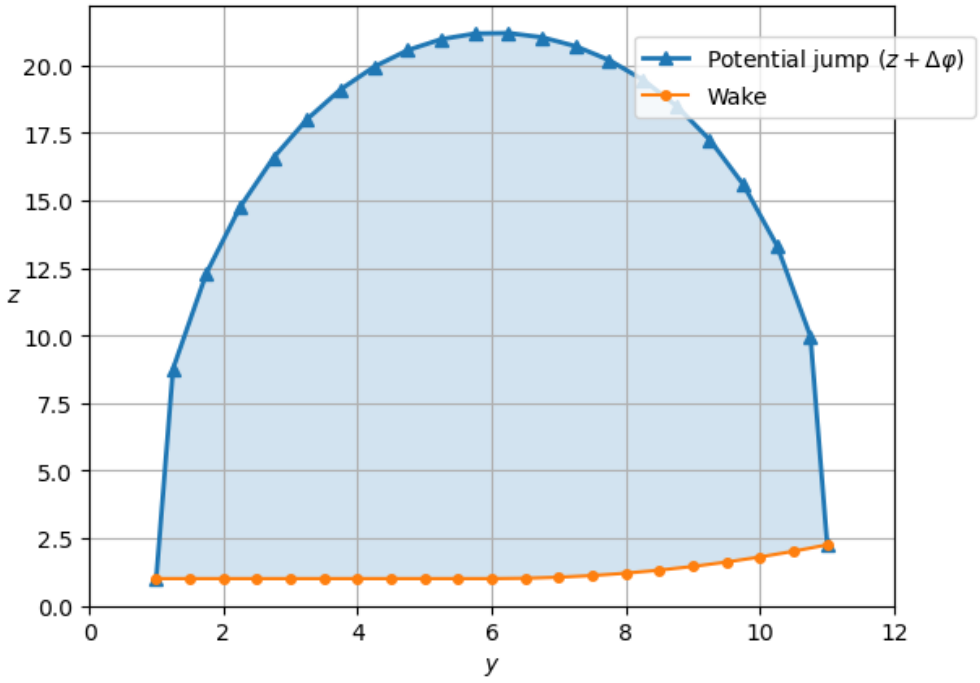

FIGURE 4.5. Lift Distribution over an asymmetric panel configuration.

4.1.2 Winglets

A second set of configurations is tested that adds winglets to the previous geometries to demonstrate more complex wing and wake shapes, shown in Figures 4.6-4.10. The winglet size is consistent with the other panels and inclined at 70 degrees from the local horizontal. The plots have the same span, with each winglet adding an extra panel. To simulate the half span of a larger aircraft, the asymmetric wake only has a winglet on the right hand edge. For the plots, the same number of panels and the same y and z shifts are used. An example with drag and potential jump output values is shown in Appendix B.

4.2 Induced Drag Convergence and Validation

For numerical validation, the SP optimization and Drela methodology are run for a range of panels for each wake geometry to compare the induced drag values and convergence. The panel ranges are selected to demonstrate practical runtimes within the scope of the LLT prob-

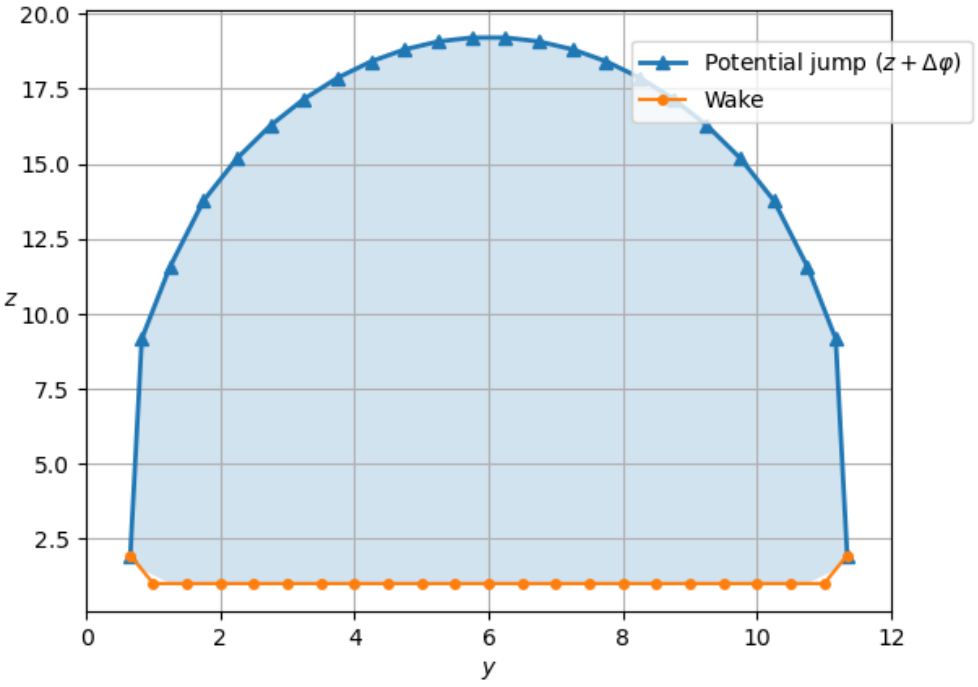

FIGURE 4.6. Lift Distribution over a flat panel configuration with winglets.

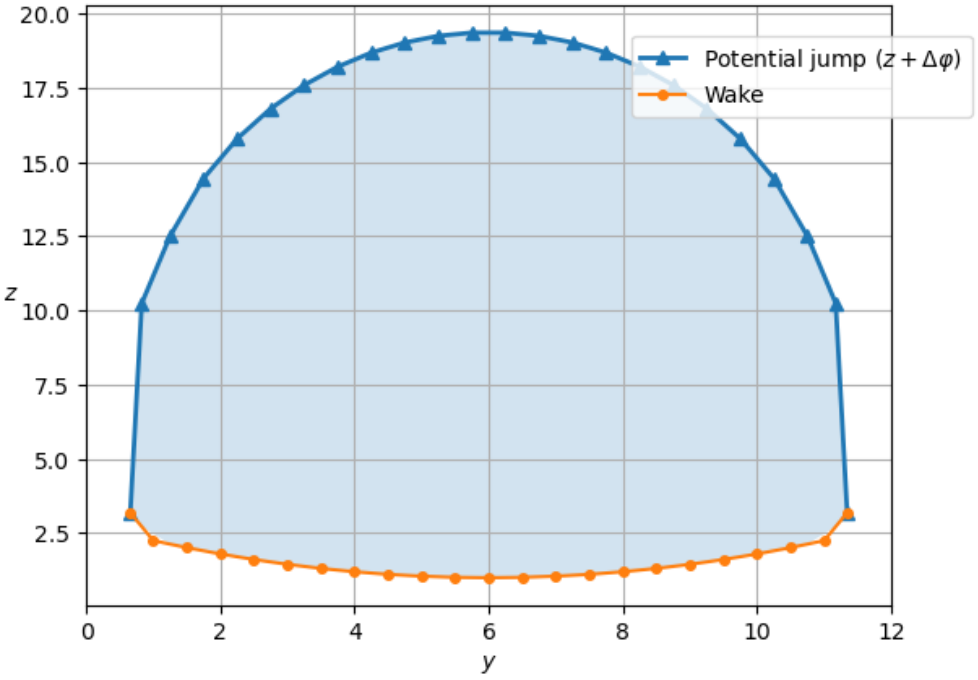

FIGURE 4.7. Lift Distribution over a parabolic panel configuration with winglets.

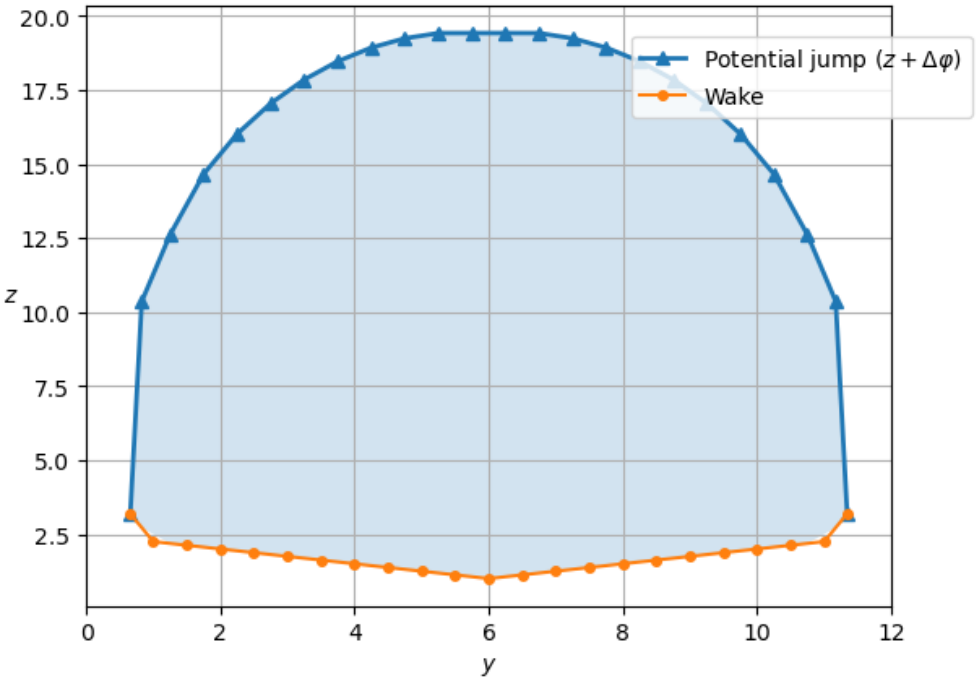

FIGURE 4.8. Lift Distribution over a linear dihedral panel configuration with winglets.

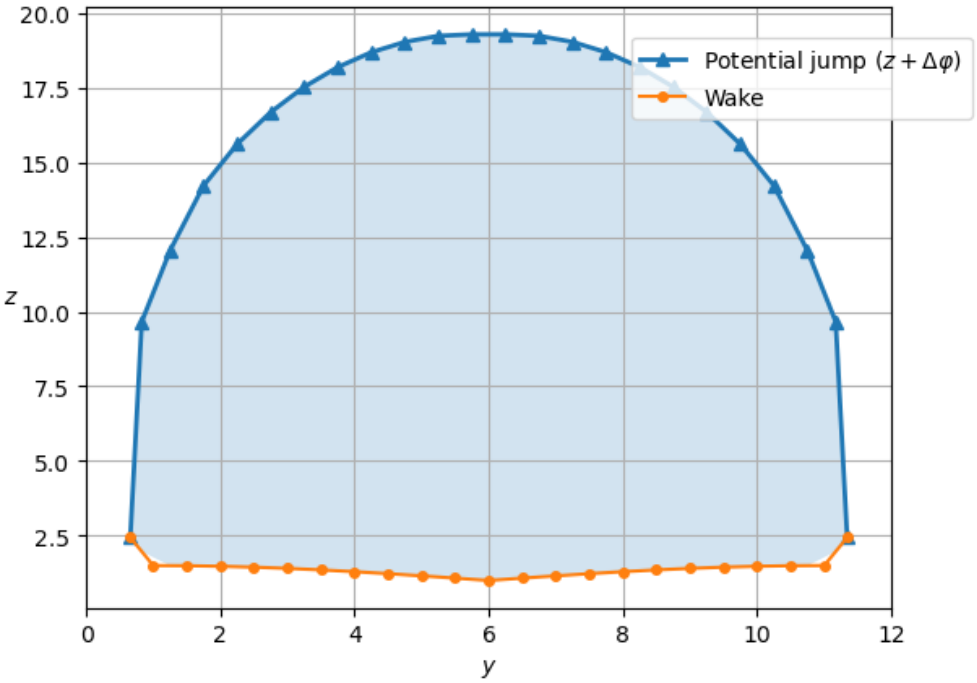

FIGURE 4.9. Lift Distribution over a sine panel configuration with winglets.

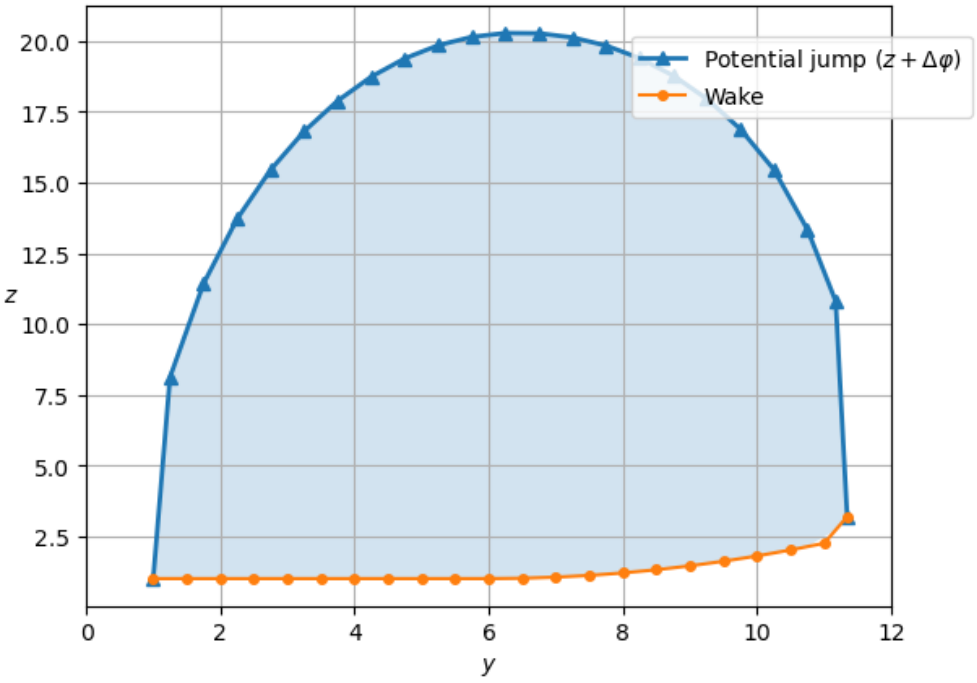

FIGURE 4.10. Lift Distribution over an asymmetric panel configuration with a winglet on the right wingtip.

lem. For the non-winglet configurations, the SP formulation runtimes notably increased when the number of panels was higher than 80, often requiring around 3 minutes to resolve. The added geometric complexity from winglets increases the SP runtime, limiting the range of panels that can be tested in a reasonable runtime. When implementing winglets, the runtime notably increases when solving for 50 panels, with a runtime around 3 minutes and 40 seconds, and increases further to a runtime over 9 minutes when solving for 60 panels.

The Drela method runtimes remained less than 10 seconds through the same ranges and configurations, as it is solving a simple linear system and equations for an optimum rather than finding the optimum with a solver. To keep consistency, the codes were run with a range of 10 to 100 panels for non-winglet configurations, and a range of 10 to 60 panels for winglet configurations, all in increments of 10. Figures 4.11-4.15 show the induced drags organized by configuration, with and without winglets, found using both Python codes. Tables for the full numerical

results and optimization runtimes are located in Appendix E.

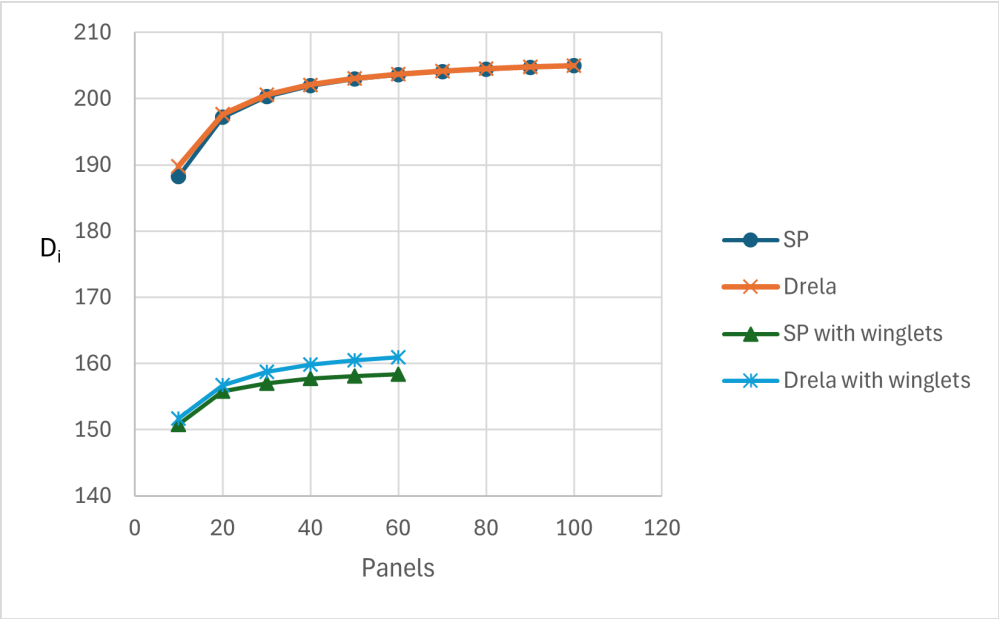

FIGURE 4.11. Induced drag as the number of panels increases for the flat panel configurations.

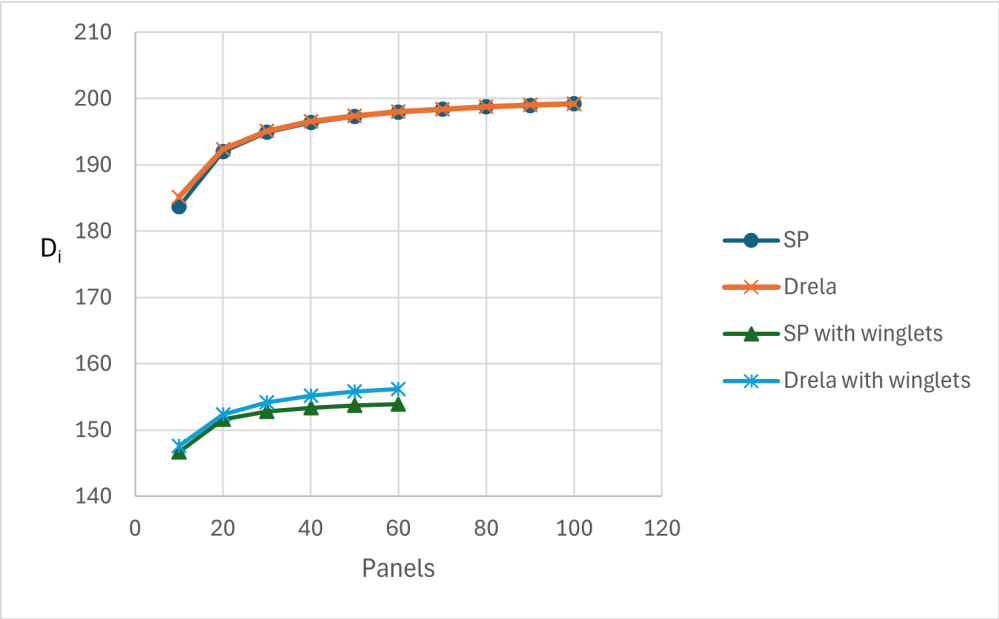

FIGURE 4.12. Induced drag as the number of panels increases for the parabolic panel configurations.

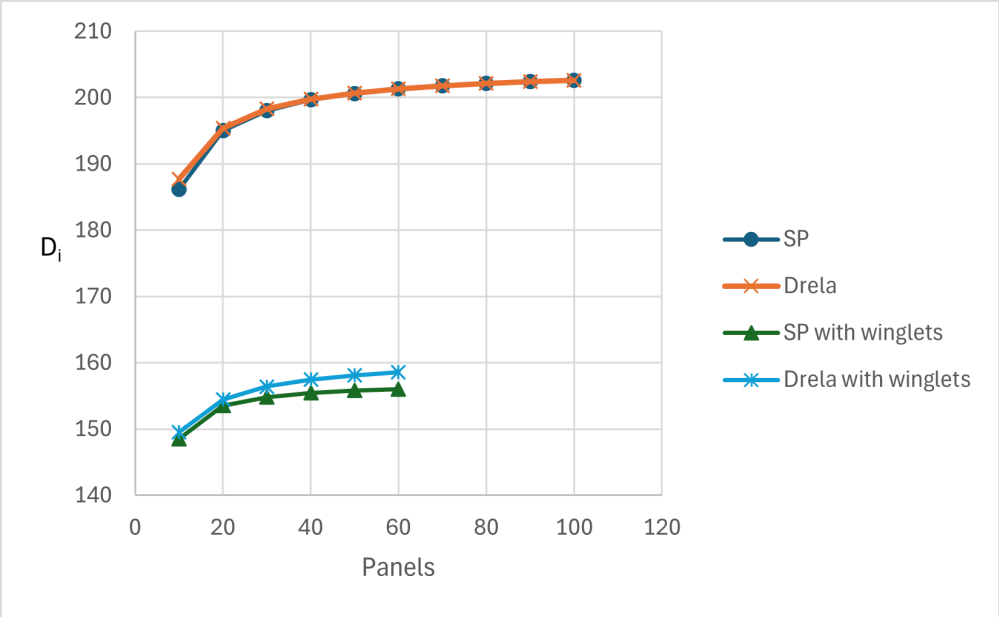

FIGURE 4.13. Induced drag as the number of panels increases for the linear dihedral panel configurations.

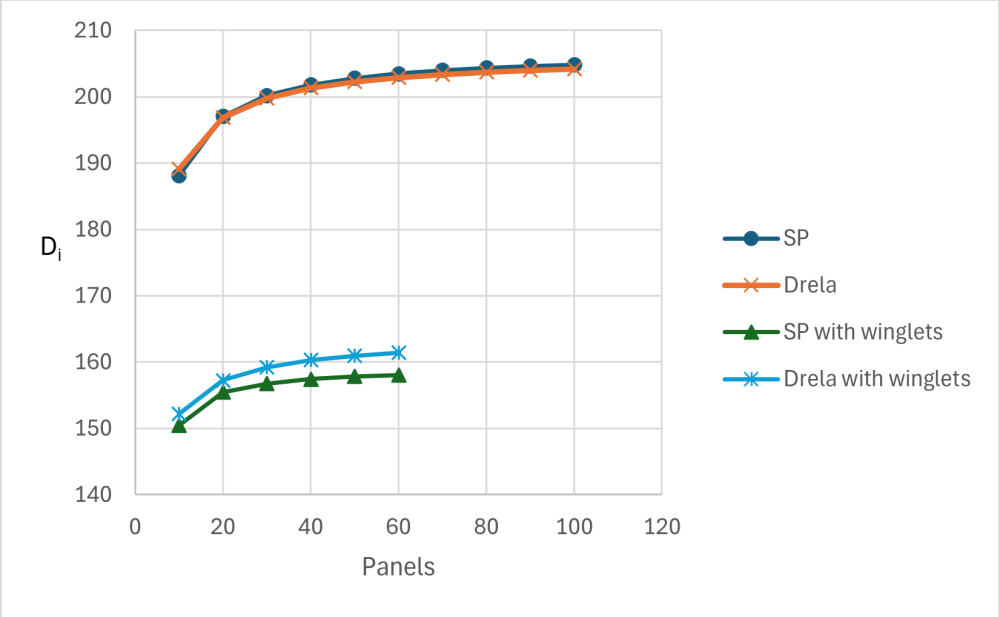

FIGURE 4.14. Induced drag as the number of panels increases for the sine panel configurations.

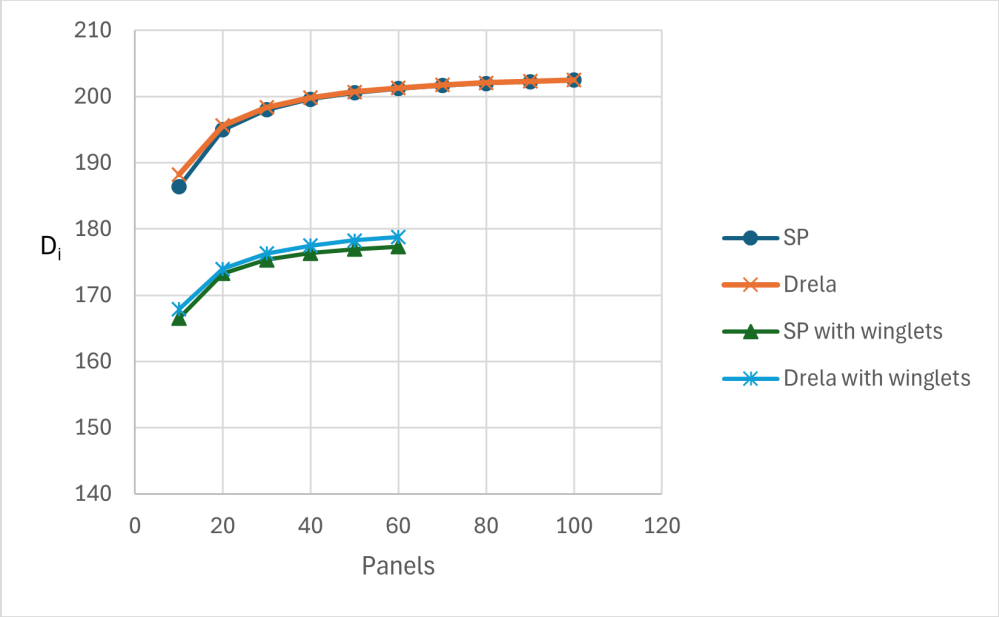

FIGURE 4.15. Induced drag as the number of panels increases for the asymmetric panel configurations.

4.2.1 Numerical Comparison

Between the Drela and SP-compatible methods, the induced drag values differed by less than 1 percent across the entire panel range for non-winglet configurations. As the number of panels increases, the percent difference generally decreases. At 50 panels, only the sine configuration had a percent difference greater than 0.1 percent. The sine configuration had a 0.28 percent difference, still sufficiently small to verify consistency in the results.

The percent differences are slightly larger in the winglet versions, typically ranging from about 0.6 percent to 1.5 percent. The sine configuration (Figure 4.14) also has the largest percent difference with winglets, around 2 percent for configurations over 50 panels. The increased differences are likely from solver tolerances and more complicated panel geometry. Adjusting solver intervals or increasing edgepoint fidelity are both possible solutions for closer matching results.

For the configurations with two winglets, induced drag is reduced by about 21 percent compared to the non-winglet configurations, roughly consistent with Whitcomb's study on winglet performance [41]. As the sole configuration with only one winglet, the asymmetric wake has

the highest induced drag. However, induced drag reduction of about 11 percent, half of the two winglet configurations, is consistent to the drag reduction ratio. The SP solver slightly undervalues the induced drag overall, but provides enough accuracy and trend information for preliminary design decisions.

4.2.2 Convergence

To visualize and compare the overall convergences for the Drela and SP compatible models, the induced drag results are combined for all winglet and non-winglet configurations, shown in Figures 4.16 and 4.17. The order of drag between each shape appears to differ from expectations, as the total panel span slightly changes between each of the configurations in order to keep the horizontal span constant. When set to the same total panel span, the flat wake results in the smallest induced drag as expected.

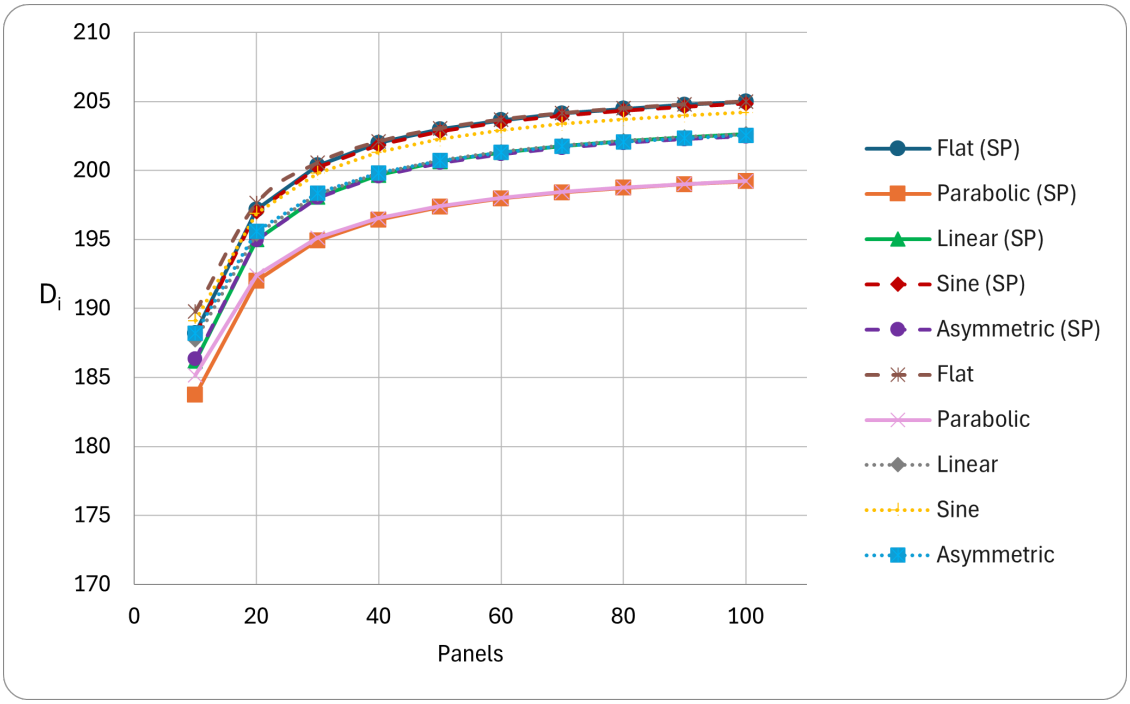

FIGURE 4.16. Induced drag convergence for all non-winglet model configurations.

Both plots follow the same trend for all geometries, converging to similar values as the number of panels increased. The most significant improvements are seen when increasing the

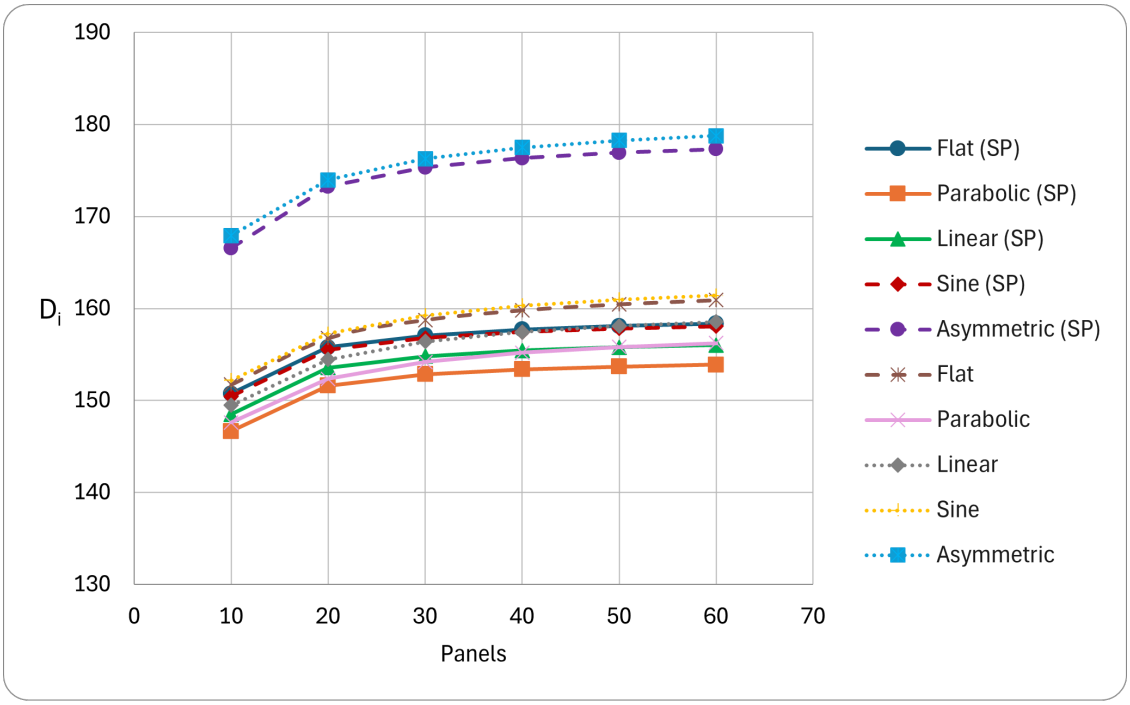

FIGURE 4.17. Induced drag convergence for all winglet model configurations.

number of panels from 10 to 20, and from 20 to 30. For winglet configurations, the difference in SP solver runtimes was small between 30 and 40 panel configurations, with noticeably larger increases when increasing from 20 to 30 panels or from 40 to 50 panels. Thus, either 30 or 40 panels is the best size for combining accuracy and runtime when including winglets. Similarly for non-winglet configurations, setting the problem to 40 panels achieves results that are sufficiently close to the actual value without requiring runtimes beyond 20 seconds.

4.3 Discussion of Results

For both formulations, with and without winglets, the range of about 30 or 40 panels provided the best balance between accuracy and runtime. At lower panel counts the wake geometry discretization produced noticeable changes in induced drag values. and at higher panel counts produced marginal accuracy improvements relative to the increase in computational cost and runtime. While a small discrepancy remains, the data is sufficient for analyzing trends and making design decisions.

Overall, induced drag values and convergence trends match between the two methods, indicating strong agreement between the SP formulation and AIC matrix solution methods. This consistency confirms the effectiveness of solving the LLT induced drag optimization within the SP framework, preserving the physics while enabling optimization. The main value of the LLT model is in preliminary design analysis, either as a standalone tool or with integration into larger multidisciplinary SP aircraft design models.

CHAPTER 5

CONCLUSIONS AND OUTLOOK

5.1 Future Work

The next step in developing this work is implementing the Lifting-Line Theory model with other Signomial Programming-compatible aircraft formulations. Of particular interest is the work by Kirschen [5], which is a large system-level aircraft design using equations from the prominent Transport Aircraft System Optimization (TASOPT) [42] at a similar level of fidelity. While the present LLT implementation has slower runtime than the Drela LLT script, the overall SP aircraft model will run faster than traditional MDO methods.

By expanding the aircraft model within the SP framework, the distinctive strengths of SP can be exploited to address a more comprehensive set of design components and constraints. This integrated formulation improves exploration into tradeoff and sensitivity relationships. Overall, the LLT integration advances the feasibility of adjustable, computationally efficient preliminary aircraft design within an SP framework.

An additional improvement for SP aircraft models would be implementations using Pyomo. The model by Kirschen [5], for example, uses the GPkit geometric programming toolkit. While an effective geometric programming tool, GPkit has is limited in its ability to implement nonlinear constraints and complex subsystem coupling. GPkit also does not have active developer support, limiting future application potential. Transitioning implementations into Pyomo will allow developers access to the wide selection of libraries and design tools for improved flexibility in SP design. Notably, the work by Avila [43] reformulates the model by Kirschen within the Pyomo framework, making the incorporation the LLT model a natural next step for an SP compatible aircraft design model.

A crucial area for improvement in the current LLT model is fidelity at the wing tips. Optimizer sensitivity can be adjusted for more accurate solutions. The Drela code or other known lifting-line methodologies and be used as benchmarks for completely matching results. Confi-

dence in higher fidelity solutions will allow for more complicated geometries and winglets, and thus testing for more complex designs.

5.1.1 Profile Drag

For the LLT formulation solving for a constant wake, profile drag was not a necessary constraint and thus not used. However, for a robust total drag model, the profile drag would be added to the formulation. The lift coefficient c_ℓ and Reynolds number Re_c are calculated through standard aerodynamic relations.

$$\boldsymbol{c_\ell} = \frac{\boldsymbol{L'}}{\frac{1}{2}\rho_\infty V_\infty^2 \boldsymbol{c}} \tag{5.1}$$

$$C_L = \frac{L}{\frac{1}{2}\,\rho_\infty\, V_\infty^2\, c\, b} \tag{5.2}$$

$$\boldsymbol{Re_c} = \frac{\rho_\infty V_\infty \boldsymbol{c}}{\mu} \tag{5.3}$$

The drag coefficient is a sum of induced, profile, and fuselage drag components.

$$C_D \geq C_{D_{\text{fus}}} + C_{D_p} + C_{D_i} \tag{5.4}$$

$$C_{D_{\text{fus}}} = \frac{0.05}{S} \tag{5.5}$$

The profile drag coefficient c_{D_p} is found with a drag model, with an example provided from the Hoburg and Abeel aircraft GP [6]. The model is in terms of the lift coefficient, Re_c, and airfoil thickness-to-chord ratio τ.

$$\begin{aligned} 1 \geq \Big(&2.56\, C_L^{5.88} \tau^{-3.32} \text{Re}_c^{-1.54} C_{D_p}^{-2.26} + 3.80 \times 10^{-9}\, C_L^{-0.92} \tau^{6.23} \text{Re}_c^{-1.38} C_{D_p}^{-9.57} \\ &+2.20 \times 10^{-3}\, C_L^{-0.01} \tau^{0.03} \text{Re}_c^{0.14} C_{D_p}^{-0.73} + 1.19 \times 10^{4}\, C_L^{9.78} \tau^{1.76} \text{Re}_c^{-1.00} C_{D_p}^{-0.91} \\ &+6.14 \times 10^{-6}\, C_L^{6.53} \tau^{-0.52} \text{Re}_c^{-0.99} C_{D_p}^{-5.19} \Big) \end{aligned} \tag{5.6}$$

Using the equation for induced drag (Equation 3.27) along with standard relations between the

drag coefficient and drag, the induced drag coefficient is

$$C_{D_i} = \frac{1}{V_\infty^2 S} \sum_{i=1}^{N} \Delta\varphi_i \frac{\partial \varphi}{\partial n_i} \Delta s_i \tag{5.7}$$

where S is the wing area. Profile drag is solved over each panel using standard relations and then solved as a summation over the span:

$$D_{p_i} \geq \frac{1}{2} \rho_\infty V_\infty^2 \boldsymbol{c c_d}$$

$$D_p \geq \sum_{i=1}^{N} D_{p_i} \Delta s_i$$

For a simple profile drag model, the standard relations are solved over each panel

$$D'_p(y) = \frac{1}{2} \rho_\infty V_\infty^2 \, c(y) \, c_d(y) \tag{5.8}$$

and the total profile drag found with a summation over the panels

$$D_p = \sum_{i=1}^{N} D'_{p_i} \, \Delta s_i \tag{5.9}$$

It is noted that the profile drag relations from [36] provide higher accuracy but would require the momentum defect of the wake along the transverse length of the wake sheet. The simplified profile drag equations are sufficient for the scope of general cases, similar to those explored in this work.

5.1.2 Variable Wake

Another path for improvement is optimizing the given wake shape. Setting the wake geometry coordinates y_i and z_i as Optimization Variables would allow the solver to manipulate the wing geometry to further minimize the induced drag. The geometry changes can be limited and

adjusted to some desired size. Because y_i and z_i are variables, they must be nonzero and positive, requiring the positive coordinate shift from Section 4.1.

When optimizing the variable wake geometry, the minimum induced drag is also found to be half of the profile drag, simplifying profile drag implementation and verification. This relationship is explained when solving the total drag sum as an optimization problem, and worked out in Appendix C. Research on optimizing wake shapes could provide insight into wing shapes and extensions that minimize induced drag, and possible optimization paths given some initial wake configuration [44].

However, the variable wake may increase the computing power and runtime beyond the practical limits for SP solvers. In constructing the constant wake LLT SP model, the author created a variable wake formulation code, shown in Appendix C, but currently available SP solvers were unable to solve beyond five panels.

Variable geometry causes the problem to become several times larger because of the prevalence of wake coordinates throughout the formulation. Direct substitution can be used to ensure SP compatibility but also increases the size of the Optimization Variables being solved. The corresponding geometry Δs_i, $\hat{n}_{iy}$, and $\hat{n}_{iz}$ are now directly affected by the changing values of y_i and z_i. Because the wake geometry is not constant, the $\hat{n}_{iy}$ and $\hat{n}_{iz}$ constraints are no longer constant. However, it is possible for the $\hat{n}_i$ values to change signs, making them SP incompatible. Thus, they must be directly substituted. The $\frac{\partial \varphi}{\partial n_i}$ expression terms can change signs. The expression is now directly substituted into the induced drag equation, since the induced drag is guaranteed to be positive by definition in the SP model. Each direct substitution causes the corresponding variable to become larger, making convergence more difficult to solve.

5.2 Conclusion

This work demonstrated that a lifting-line theory formulation can be constructed as a Signomial Programming framework, allowing for the expression of aerodynamic mathematics into

an SP Multidisciplinary Optimization architecture. The lifting-line formulation was based on a discrete potential jump and panel representation, with the objective of minimizing induced drag. For validation, a second code was developed that constructed an Aerodynamic Influence Coefficient matrix to directly compute the optimal potential jumps and lift distribution.

The models were tested using five different wake configurations: flat, parabolic, linear dihedral, sine, and asymmetric. Winglets were added at the tips for each of the configurations for a total of ten testing configurations. The computed lift distributions reproduced the classical elliptical trends for symmetric configurations and demonstrated expected deviations for asymmetric cases. Induced drag calculations followed expected theoretical scaling behavior, with and without winglets. Convergence trends were also consistent across configurations, with small differences observed between the winglet and non-winglet cases. Overall, results confirmed consistency of the SP formulation with the AIC matrix solution.

The results from this work add to the foundation for multidisciplinary SP implementations in aircraft design, leveraging problem structures to reduce computational time and cost while retaining fidelity. Further research into broadening SP modeling and tightened multidisciplinary optimization will give further insight into the practical limits and advantages of such methods. Fundamentally, these efforts will aid design engineers expand their explorations into novel configurations, performance trade spaces, and design optimizations of the future.

APPENDICES

APPENDIX A

PYTHON CODES FOR LIFTING-LINE MODELS

```
# LIFTING LINE THEORY SIGNOMIAL PROGRAMMING VERSION

# Import statements
import numpy as np
import pyomo
from edi import Formulation
from pyomo.environ import units
import matplotlib.pyplot as plt
%matplotlib inline

# Set up geometry ##########################
GEOMETRY = "parabolic" # options: flat, parabolic, linear, sine, cubic, exponential, asymmetric
WINGLETS = "none"   # options: "none", "left", "right", "both"
FLIP = False # options: True, False (flip the sign of the curves ie dihedral vs anhedral)

y_half = np.linspace(1,11,21) # start, end, # of points (including endpoints)
z_half = []
############################################

# flat at z=0
if GEOMETRY == "flat":
    z_half = 0*y_half

# parabolic
elif GEOMETRY == "parabolic":
    z_half = 0.05*(y_half-(y_half[-1]-y_half[0])/2 - y_half[0])**2

# linear with dihedral (angle is arctan of coefficient in z_half eqn)
elif GEOMETRY == "linear":
    span = y_half[-1] - y_half[0]
    z_half = 0.25 * np.abs(y_half - (y_half[0] + span/2))

# sine wave (absolute value)
elif GEOMETRY == "sine":
    for i in range(len(y_half) // 2 + 1):
        z_half.append(0.5 * np.sin((y_half[i] - y_half[0]) * np.pi / (y_half[-1] - y_half[0])))
    z_half = np.concatenate([z_half[::-1], z_half[1:]])

# cubic
elif GEOMETRY == "cubic":
    for i in range(len(y_half) // 2 + 1):
        z_half.append(0.1 * (y_half[i] - (y_half[-1] + y_half[0]) / 2) ** 3)
    z_half = np.concatenate([z_half[::-1], z_half[1:]])

# exponential
elif GEOMETRY == "exponential":
    for i in range(len(y_half) // 2 + 1):
        z_half.append(0.1 * np.exp(y_half[i] - y_half[0]))
    z_half = np.concatenate([z_half[::-1], z_half[1:]])

# split into asymmetric halves
elif GEOMETRY == "asymmetric": # currently left linear, right parabolic
    mid = len(y_half)//2
```

```
    y_left  = y_half[:mid+1]
    y_right = y_half[mid+1:]

    # Left: linear
    z_left = y_left*0 # linear at z=0

    # Right: parabolic
    z_right = 0.05*(y_right - y_left[-1])**2 + z_left[-1]

    z_half = np.concatenate([z_left, z_right])

else :
    raise ValueError("Invalid geometry option. Choose from: flat, parabolic, linear, sine, cubic, exponential, asymmetric.")

# shift along z
z_half = z_half - np.min(z_half) + 1

# flip the sign/curves (dihedral/anhedral) ###############
if FLIP:
    z_half = -z_half
    # Shift so lowest point is zero
    z_half = z_half - np.min(z_half) + 1

# plotting winglets ##########################
if WINGLETS != "none":

    y_left  = y_half[0]
    y_right = y_half[-1]
    z_left  = z_half[0]
    z_right = z_half[-1]

    winglet_len = 1
    n_winglet_pts = 1
    cant_angle = np.deg2rad(70)

    ds_w = np.linspace(0, winglet_len, n_winglet_pts+1)[1:]

    dy = ds_w * np.cos(cant_angle)
    dz = ds_w * np.sin(cant_angle)

    # add left winglet
    if WINGLETS in ["left", "both"]:
        y_winglet_left = y_left - dy
        z_winglet_left = z_left + dz

        y_half = np.concatenate([y_winglet_left[::-1], y_half])
        z_half = np.concatenate([z_winglet_left[::-1], z_half])

    # add right winglet
    if WINGLETS in ["right", "both"]:
        y_winglet_right = y_right + dy
        z_winglet_right = z_right + dz

        y_half = np.concatenate([y_half, y_winglet_right])
```

```
        z_half = np.concatenate([z_half, z_winglet_right])

print(y_half)
print(z_half)
y = (y_half[1:] + y_half[0:-1])/2
z = (z_half[1:] + z_half[0:-1])/2

Delta_s = ( (y_half[1:] - y_half[0:-1])**2 + (z_half[1:] - z_half[0:-1])**2 )**0.5
print(Delta_s)
n_hat_y = -(z_half[1:] - z_half[0:-1])/Delta_s
n_hat_z = (y_half[1:] - y_half[0:-1])/Delta_s
print(n_hat_y)
print(n_hat_z)


# Formulation
f = Formulation()

# Variables
Delta_phi  = f.Variable(name='Delta_phi' , guess=1.0, units='', size=len(y), description='The potentials')
dphi_dn    = f.Variable(name='dphi_dn'   , guess=1.0, units='', size=len(y), description='potential grad normals')
D_i        = f.Variable(name='D_i'       , guess=1.0, units='',              description='Induced Drag')

# Constants
L_spec     = f.Constant(name='L_spec', value=10000.0, units='', description='Specified Lift')
rho        = f.Constant(name='rho'   , value=1.23   , units='', description='Density')
V          = f.Constant(name='V'     , value=50.    , units='', description='Velocity')


f.Objective(D_i)
# should be D_i, if different then change back

# Constraints
constraints = []

temp_holder = 0
for i in range(0, len(y)):
    temp_holder += Delta_phi[i] * (y_half[i+1]-y_half[i])
constraints += [L_spec == rho * V * temp_holder,
                ]

for i in range(0, len(y)):
    temp_holder = 0
    for j in range(0, len(y_half)):
        if j == 0:
            temp_holder += (                - Delta_phi[j] ) * -(z[i]-z_half[j]) / ( (y[i]-y_half[j])**2 + (z[i]-z_half[j])**2 )
        elif j == len(y_half)-1:
            temp_holder += ( Delta_phi[j-1]                ) * -(z[i]-z_half[j]) / ( (y[i]-y_half[j])**2 + (z[i]-z_half[j])**2 )
        else:
            temp_holder += ( Delta_phi[j-1] - Delta_phi[j] ) * -(z[i]-z_half[j]) / ( (y[i]-y_half[j])**2 + (z[i]-z_half[j])**2 )
    Grad_phi_y = 1/(2*np.pi) * temp_holder

    temp_holder = 0
    for j in range(0, len(y_half)):
```

```
        if j == 0:
            temp_holder += (                - Delta_phi[j] ) * (y[i]-y_half[j]) / ( (y[i]-y_half[j])**2 + (z[i]-z_half[j])**2 )
        elif j == len(y_half)-1:
            temp_holder += ( Delta_phi[j-1]                ) * (y[i]-y_half[j]) / ( (y[i]-y_half[j])**2 + (z[i]-z_half[j])**2 )
        else:
            temp_holder += ( Delta_phi[j-1] - Delta_phi[j] ) * (y[i]-y_half[j]) / ( (y[i]-y_half[j])**2 + (z[i]-z_half[j])**2 )
    Grad_phi_z = 1/(2*np.pi) * temp_holder


    constraints += [dphi_dn[i] >= -1*(Grad_phi_y * n_hat_y[i] + Grad_phi_z * n_hat_z[i]) ]

temp_holder = 0
for i in range(0, len(y)):
    temp_holder += Delta_phi[i] * dphi_dn[i] * Delta_s[i]
constraints += [D_i >= 1/2 * rho * temp_holder,

                ]

f.ConstraintList(constraints)

# from solver import cvxopt_solve
from edi.solvers.solver import cvxopt_solve
res = cvxopt_solve(f)
print(res['x'])

vars = f.get_variables()
print([str(v) for v in vars])
post_Delta_phi = res['x'][0:len(y)]
post_Delta_phi = np.array(post_Delta_phi).T[0]
y_with_ends = np.append(np.append(y_half[0],y),y_half[-1])
z_with_ends = np.append(np.append(z_half[0],z),z_half[-1])
dp_with_ends = np.append(np.append(0,post_Delta_phi),0)
plt.plot(y_with_ends, dp_with_ends+z_with_ends, '^-', lw=2, label=r'Potential jump ($z + \Delta\varphi$)')
# plt.axis('equal')

plt.fill_between(y_with_ends, z_with_ends, dp_with_ends+z_with_ends, alpha=0.2)
plt.xlabel(r'$y$')
plt.xlim(0, 12) # change based on span
plt.ylabel(r'$z$', rotation=0)
plt.plot(y_half, z_half, 'o-', ms=4, label='Wake')
plt.legend(loc='upper right', bbox_to_anchor=(1.1, 0.95), borderaxespad=0)
plt.grid(True)


# Display code
import pandas as pd
from pyomo.core.base.var import Var, IndexedVar

var_list = f.get_variables()
con_list = f.get_constants()

variable_names = []
variable_values = []
variable_units = []
```

```
variable_description = []

constant_names = []
constant_values = []
constant_units = []
constant_description = []

ctr = 0
for var in var_list:
    if isinstance(var, IndexedVar):
        for ix in var.index_set():
            variable_names.append(f"{var.name}[{ix}]")   # works for tuples too
            variable_values.append(res['x'][ctr])
            variable_units.append(getattr(var[ix], "_units", None))
            variable_description.append(getattr(var[ix], "doc", None))
            ctr += 1
    else:
        variable_names.append(var.name)
        variable_values.append(res['x'][ctr])
        variable_units.append(getattr(var, "_units", None))
        variable_description.append(getattr(var, "doc", None))
        ctr += 1

pd.set_option('display.max_rows', None)
pd.options.display.float_format = '{:.4f}'.format
vdf = pd.DataFrame({
    'Variable Name': variable_names,
    'Value': variable_values,
    'Units': variable_units,
    'Description': variable_description
})

print(vdf)

# Handle constants
for con in con_list:
    constant_names.append(con.name)
    constant_values.append(con.value)
    constant_units.append(getattr(con, "_units", None))
    constant_description.append(getattr(con, "doc", None))

pd.set_option('display.max_rows', None)
pd.options.display.float_format = '{:.4}'.format
cdf = pd.DataFrame({
    'Constant Name': constant_names,
    'Value': constant_values,
    'Units': constant_units,
    'Description': constant_description
})

print(cdf)

# LIFTING LINE THEORY DRELA VERSION ##############################

# Import Statements
```

```python
import numpy as np
import matplotlib.pyplot as plt

# Constants
rho = 1.23
V = 50.0
L_spec = 10000  # desired total lift, specified lift in SP

# Set up geometry
N = 40 # panels
ymin = 1.0
ymax = 11
y_half = np.linspace(ymin, ymax, N+1)

GEOMETRY = "parabolic" # options: flat, parabolic, linear, sine, cubic, exponential, asymmetric
WINGLETS = "none"   # options: "none", "left", "right", "both"
FLIP = False # options: True, False (flip the sign of the curves ie dihedral vs anhedral)

# flat at z=0
if GEOMETRY == "flat":
    z_half = np.zeros_like(y_half)

# parabolic
elif GEOMETRY == "parabolic":
    z_half = 0.05*(y_half-(y_half[-1]-y_half[0])/2 - y_half[0])**2

# linear with dihedral (angle is arctan of coefficient in z_half eqn)
elif GEOMETRY == "linear":
# linear increase in z over half the span
    half_len = len(y_half)//2 + 1
    z_half_first = 0.25 * (y_half[:half_len] - y_half[0])
    # mirror about midspan
    z_half = np.concatenate([z_half_first[::-1], z_half_first[1:]])

# sine wave (absolute value)
elif GEOMETRY == "sine":
    # sine wave (absolute value)
    span = y_half[-1] - y_half[0]
    z_half = 0.5 * np.sin(np.pi * (y_half - y_half[0]) / span)

# cubic
elif GEOMETRY == "cubic":
    for i in range(len(y_half) // 2 + 1):
        z_half.append(0.1 * (y_half[i] - (y_half[-1] + y_half[0]) / 2) ** 3)
    z_half = np.concatenate([z_half[::-1], z_half[1:]])

# exponential
elif GEOMETRY == "exponential":
    for i in range(len(y_half) // 2 + 1):
        z_half.append(0.1 * np.exp(y_half[i] - y_half[0]))
    z_half = np.concatenate([z_half[::-1], z_half[1:]])

# split into asymmetric halves
elif GEOMETRY == "asymmetric": # currently left linear, right parabolic
```

```
    mid = len(y_half)//2

    y_left  = y_half[:mid+1]
    y_right = y_half[mid+1:]

    # Left: linear
    z_left = y_left*0 # linear at z=0

    # Right: parabolic
    z_right = 0.05*(y_right - y_left[-1])**2 + z_left[-1]

    z_half = np.concatenate([z_left, z_right])

else :
    raise ValueError("Invalid geometry option. Choose from: flat, parabolic, linear, sine, cubic, exponential, asymmetric.")

# shift along z
z_half = np.min(z_half) + 1

# flip the sign/curves (dihedral/anhedral) ###############
if FLIP:
    z_half = -z_half
    # Shift so lowest point is zero
    z_half = z_half - np.min(z_half) + 1

# plotting winglets ##########################
if WINGLETS != "none":

    y_left  = y_half[0]
    y_right = y_half[-1]
    z_left  = z_half[0]
    z_right = z_half[-1]

    winglet_len = 1
    n_winglet_pts = 1
    cant_angle = np.deg2rad(70)

    ds_w = np.linspace(0, winglet_len, n_winglet_pts+1)[1:]

    dy = ds_w * np.cos(cant_angle)
    dz = ds_w * np.sin(cant_angle)

    # add left winglet
    if WINGLETS in ["left", "both"]:
        y_winglet_left = y_left - dy
        z_winglet_left = z_left + dz

        y_half = np.concatenate([y_winglet_left[::-1], y_half])
        z_half = np.concatenate([z_winglet_left[::-1], z_half])

    # add right winglet
    if WINGLETS in ["right", "both"]:
        y_winglet_right = y_right + dy
        z_winglet_right = z_right + dz
```

```
        y_half = np.concatenate([y_half, y_winglet_right])
        z_half = np.concatenate([z_half, z_winglet_right])

############################################################
# Control points
y = (y_half[1:] + y_half[:-1])/2
z = (z_half[1:] + z_half[:-1])/2
ds = ( (y_half[1:]-y_half[:-1])**2 + (z_half[1:]-z_half[:-1])**2 )**0.5

def drela(rho, V, y, z, y_half, z_half, L_spec):

    N = len(y)

    ny = -(z_half[1:] - z_half[0:-1])/ds
    nz = (y_half[1:] - y_half[0:-1])/ds

    # AIC matrix setup
    C = np.zeros((N, N+1))

    for i in range(N):
        for j in range(N+1):

            dyij = y[i] - y_half[j]
            dzij = z[i] - z_half[j]

            denom = dyij**2 + dzij**2

            Cy = -dzij / denom
            Cz =  dyij / denom

            C[i,j] = (ny[i]*Cy + nz[i]*Cz) / (2*np.pi)

    D = np.zeros((N+1, N)) # applying bc and interior points

    D[0,0] = -1     # keeping signs consistent with delta phis in SP
    for j in range(1, N):
        D[j,j-1] =  1
        D[j,j]   = -1
    D[N,N-1] = 1

    A = C @ D # aic matrix

    cos_theta = (y_half[1:] - y_half[:-1]) / ds

    # # bvec based on wake geometry
    # bvec = cos_theta
    # delta_phi_shape = np.linalg.solve(A, bvec) # solve aic matrix
    # L_current = rho * V * np.sum(delta_phi_shape *ds*cos_theta)
    # # scale to L_spec
    # scale = L_spec / L_current
    # delta_phi = delta_phi_shape * scale

    # extending system for lambda
```

```
    A_ext = np.zeros((N+1, N+1))
    A_ext[:N,:N] = A  # first N rows
    A_ext[:N,-1] = -cos_theta  # lambda column
    A_ext[N,:N] = rho * V * ds * cos_theta  # lift constraint
    # Last column, last row = 0 (no lambda in lift sum)

    # Right side
    b_ext = np.zeros(N+1)
    b_ext[N] = L_spec

    # Solve for delta_phi and Lambda
    sol = np.linalg.solve(A_ext, b_ext)
    delta_phi = sol[:N]
    # both delta_phi methods are equivalent


    dphi_dn = A @ delta_phi
    D_i = -0.5 *rho* np.sum(delta_phi * dphi_dn * ds)

    return delta_phi, D_i


delta_phi, D_i = drela(rho, V, y, z, y_half, z_half, L_spec)

print("delta_phi:", delta_phi)
print("Induced drag:", D_i)

# Plotting
# Build arrays including endpoints
y_with_ends = np.append(np.append(y_half[0], y), y_half[-1])
z_with_ends = np.append(np.append(z_half[0], z), z_half[-1])
dp_with_ends = np.append(np.append(0, delta_phi), 0)

plt.figure(figsize=(10,6))
plt.plot(y_with_ends, dp_with_ends + z_with_ends, '^-', lw=2, label=r'$\Delta \varphi$')

# Fill between surface and lifted curve
plt.fill_between(
    y_with_ends,
    z_with_ends,
    dp_with_ends + z_with_ends,
    alpha=0.2
)

# Plot panel geometry
plt.plot(y_half, z_half, 'o-', label='Panel edges')

plt.axis('equal')
plt.xlabel('y')
plt.ylabel('z')
plt.grid(True)
plt.legend()

plt.show()
```

APPENDIX B

SAMPLE OUTPUT

The SP code is run for parabolic 30-panel configuration with winglets.

Variable Name	Value	Units	Description
Delta_phi[0]	7.792	None	None
Delta_phi[1]	9.492	None	None
Delta_phi[2]	11.42	None	None
Delta_phi[3]	12.75	None	None
Delta_phi[4]	13.81	None	None
Delta_phi[5]	14.7	None	None
Delta_phi[6]	15.45	None	None
Delta_phi[7]	16.09	None	None
Delta_phi[8]	16.64	None	None
Delta_phi[9]	17.11	None	None
Delta_phi[10]	17.5	None	None
Delta_phi[11]	17.82	None	None
Delta_phi[12]	18.07	None	None
Delta_phi[13]	18.25	None	None
Delta_phi[14]	18.37	None	None
Delta_phi[15]	18.43	None	None
Delta_phi[16]	18.43	None	None
Delta_phi[17]	18.37	None	None
Delta_phi[18]	18.25	None	None
Delta_phi[19]	18.07	None	None
Delta_phi[20]	17.82	None	None
Delta_phi[21]	17.5	None	None
Delta_phi[22]	17.11	None	None
Delta_phi[23]	16.64	None	None
Delta_phi[24]	16.09	None	None
Delta_phi[25]	15.45	None	None
Delta_phi[26]	14.7	None	None
Delta_phi[27]	13.81	None	None
Delta_phi[28]	12.75	None	None
Delta_phi[29]	11.42	None	None
Delta_phi[30]	9.492	None	None
Delta_phi[31]	7.792	None	None
dphi_dn[0]	1.226	None	None
dphi_dn[1]	0.02477	None	None
dphi_dn[2]	1.273	None	None
dphi_dn[3]	1.403	None	None
dphi_dn[4]	1.429	None	None
dphi_dn[5]	1.445	None	None
dphi_dn[6]	1.458	None	None
dphi_dn[7]	1.47	None	None

dphi_dn[8]	1.481	None	None
dphi_dn[9]	1.491	None	None
dphi_dn[10]	1.5	None	None
dphi_dn[11]	1.507	None	None
dphi_dn[12]	1.513	None	None
dphi_dn[13]	1.518	None	None
dphi_dn[14]	1.521	None	None
dphi_dn[15]	1.522	None	None
dphi_dn[16]	1.522	None	None
dphi_dn[17]	1.521	None	None
dphi_dn[18]	1.518	None	None
dphi_dn[19]	1.513	None	None
dphi_dn[20]	1.507	None	None
dphi_dn[21]	1.5	None	None
dphi_dn[22]	1.491	None	None
dphi_dn[23]	1.481	None	None
dphi_dn[24]	1.47	None	None
dphi_dn[25]	1.458	None	None
dphi_dn[26]	1.445	None	None
dphi_dn[27]	1.429	None	None
dphi_dn[28]	1.403	None	None
dphi_dn[29]	1.273	None	None
dphi_dn[30]	0.02477	None	None
dphi_dn[31]	1.226	None	None
D_i	152.8	dimensionless	Induced Drag

Constant Name	Value	Units	Description
L_spec	1e+04	dimensionless	Specified Lift
rho	1.23	dimensionless	Density
V	50.0	dimensionless	Velocity

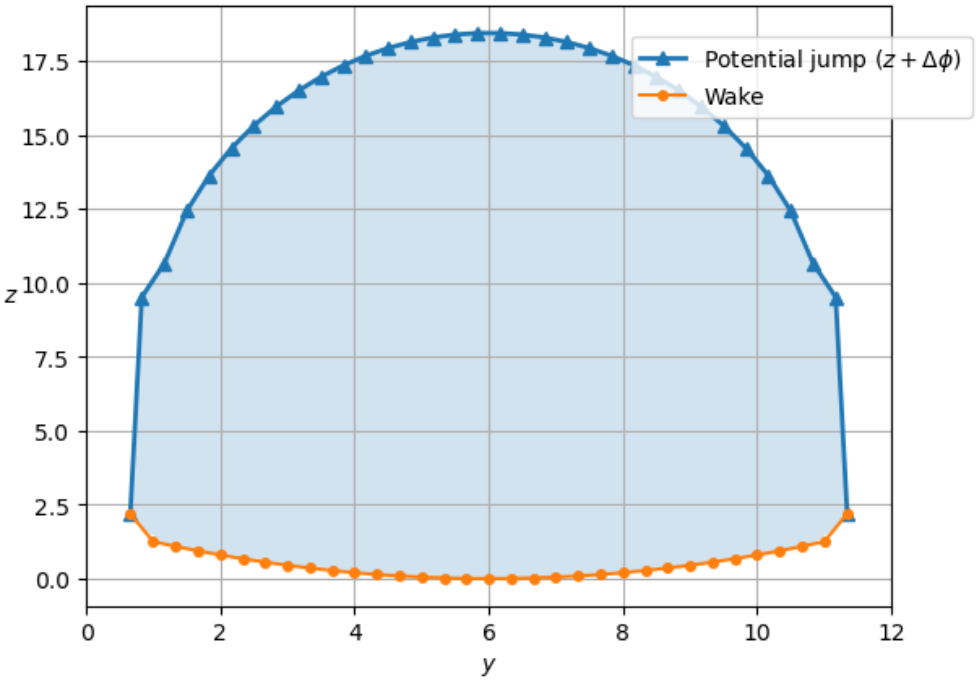

FIGURE B.1. Lift Distribution over a parabolic 30 panel configuration with winglets.

APPENDIX C

LIFTING-LINE MODEL OUTLINE FOR VARIABLE WAKE

The variable wake model is established in this section, but cannot be solved currently with available technology (i.e. EDI and CVXOPT). The code is provided as an outline to how the variable wake case is modeled.

```
# VARIABLE WAKE CASE
import numpy as np
import pyomo
from edi import Formulation
from pyomo.environ import units
import matplotlib.pyplot as plt
import math
# %matplotlib inline

n_panels = 3 # number of panels

f = Formulation()

# Variables
c_dp      = f.Variable(name='c_dp'      , guess=1.0,  units='', size=n_panels,   description='Profile drag coefficient')
D_p       = f.Variable(name='D_p'       , guess=1.0,  units='', size=n_panels,   description='Profile Drag')
D_pTotal  = f.Variable(name='D_pTotal'  , guess=1.0,  units='',                  description='Total Profile Drag')
Delta_phi = f.Variable(name='Delta_phi' , guess=1.0,  units='', size=n_panels,   description='The potentials')
Delta_s   = f.Variable(name='Delta_s'   , guess=1.0,  units='', size=n_panels,   description='Panel length along wake horizontal')
D_i       = f.Variable(name='D_i'       , guess=1.0,  units='',                  description='Induced Drag')
y         = f.Variable(name='y'         , guess=1.0,  units='', size=n_panels,   description='y coordinate')
y_half    = f.Variable(name='y_half'    , guess=1.0,  units='', size=n_panels+1, description='y half coordinate')
z         = f.Variable(name='z'         , guess=1.0,  units='', size=n_panels,   description='z coordinate')
z_half    = f.Variable(name='z_half'    , guess=1.0,  units='', size=n_panels+1, description='z half coordinate')
x         = f.Variable(name='x'         , guess=1.0,  units='', size=(n_panels, n_panels+1), description='intermediate distance term')
S_ref     = f.Variable(name='S_ref'     , guess=1.0,  units='',                  description='Reference Area')
L_prime   = f.Variable(name='L_prime'   , guess=1.0,  units='', size=n_panels,   description='Lift per unit span')
c_l       = f.Variable(name='c_l'       , guess=1.0,  units='', size=n_panels,   description='Lift coefficient')
V         = f.Variable(name='V'         , guess=15.0, units='',                  description='Velocity')
rho       = f.Variable(name='rho'       , guess=1.23, units='',                  description='Density')

# Constants
c      = f.Constant(name='c',      value=0.35,   units='', description='Chord length')
L_spec = f.Constant(name='L_spec', value=1000.0, units='', description='Specified Lift')


f.Objective(D_i+D_pTotal)

constraints = []
constraints += [
                rho <= 1.23,
                y_half[0] == 1,
                ]

for i in range(0, n_panels+1):
    constraints += [
                    z_half[int(math.floor((n_panels)/2))] == 1,
                    ]
```

```
for i in range(0, n_panels):
    constraints += [
                    y_half[i+1] - y_half[i] >= 0.1, # placeholder constraint so panels don't overlap

                    y[i] == (y_half[i] + y_half[i+1])/2,
                    z[i] == (z_half[i] + z_half[i+1])/2,

                    Delta_s[i]**2 == (y_half[i+1] - y_half[i])**2 + (z_half[i+1] - z_half[i])**2,

                    L_prime[i] == Delta_phi[i]*(y_half[i+1]-y_half[i]),
                    c_l[i] == L_prime[i] / (0.5*rho*V**2*c),

                    c_dp[i] >= 0.02 + 0.1*c_l[i]**2, # placeholde approximation
                    D_p[i] >= 0.5*rho*(V**2)*c_dp[i]*c*Delta_s[i],
                    ]
temp_holderS = 0
for i in range(0, n_panels):
    temp_holderS += Delta_s[i]
constraints += [S_ref == c * temp_holderS]

temp_holder0 = 0
for i in range(0, n_panels):
    temp_holder0 += D_p[i]
constraints += [D_pTotal >= temp_holder0]

for i in range(0, n_panels-1):
    constraints += [
                    Delta_s[i] == Delta_s[i+1], # enforce equal panel lengths
                    ]

temp_holder = 0
for i in range(0, n_panels):
    temp_holder += Delta_phi[i] * (y_half[i+1]-y_half[i])
constraints += [L_spec <= rho * V * temp_holder]


temp_holder3 = 0
temp_holder4 = 0
for i in range(0, n_panels):
    temp_holder1 = 0
    for j in range(0, n_panels+1):
        constraints += [x[i,j] == (y[i]-y_half[j])**2 + (z[i]-z_half[j])**2,
                        ]
        if j == 0:
            temp_holder1 += (                - Delta_phi[j] ) * -(z[i]-z_half[j]) / ( x[i,j] )
        elif j == n_panels:
            temp_holder1 += ( Delta_phi[j-1]                ) * -(z[i]-z_half[j]) / ( x[i,j] )
        else:
            temp_holder1 += ( Delta_phi[j-1] - Delta_phi[j] ) * -(z[i]-z_half[j]) / ( x[i,j] )

    temp_holder2 = 0
    for j in range(0, n_panels+1):
        if j == 0:
            temp_holder2 += (                - Delta_phi[j] ) * (y[i]-y_half[j]) / ( x[i,j] )
```

```
        elif j == n_panels:
            temp_holder2 += ( Delta_phi[j-1]                ) * (y[i]-y_half[j]) / ( x[i,j] )
        else:
            temp_holder2 += ( Delta_phi[j-1] - Delta_phi[j] ) * (y[i]-y_half[j]) / ( x[i,j] )

    dphi_dny = 1/(2*np.pi)* temp_holder1 * -(z_half[i+1] - z_half[i])/Delta_s[i]
    dphi_dnz = 1/(2*np.pi)* temp_holder2 * (y_half[i+1] - y_half[i])/Delta_s[i]
    temp_holder3 = -(dphi_dny+dphi_dnz)*Delta_phi[i]*Delta_s[i]
    temp_holder4 += temp_holder3


constraints += [
                D_i >= 0.5*rho*temp_holder4,
                ]


f.ConstraintList(constraints)

var_list = f.get_variables()
for v in var_list:
    if isinstance(v,pyomo.core.base.var.IndexedVar):
        for ix in v.index_set():
            f.Constraint( v[ix] >= 1e-8*v._units )
            f.Constraint( v[ix] <= 1e8*v._units )
    else:
        f.Constraint( v >= 1e-8*v._units )
        f.Constraint( v <= 1e8*v._units )
# f.pprint()

vars = f.get_variables()

# from solver import cvxopt_solve
from edi.solvers.solver import cvxopt_solve
res = cvxopt_solve(f)
print(res['x'])

vars = f.get_variables()
print([str(v) for v in vars])

# Display code
import pandas as pd
from pyomo.core.base.var import Var, IndexedVar

var_list = f.get_variables()
con_list = f.get_constants()

variable_names = []
variable_values = []
variable_units = []
variable_description = []

constant_names = []
constant_values = []
constant_units = []
```

```
constant_description = []

ctr = 0
for var in var_list:
    if isinstance(var, IndexedVar):
        for ix in var.index_set():
            variable_names.append(f"{var.name}[{ix}]")    # works for tuples too
            variable_values.append(res['x'][ctr])
            variable_units.append(getattr(var[ix], "_units", None))
            variable_description.append(getattr(var[ix], "doc", None))
            ctr += 1
    else:
        variable_names.append(var.name)
        variable_values.append(res['x'][ctr])
        variable_units.append(getattr(var, "_units", None))
        variable_description.append(getattr(var, "doc", None))
        ctr += 1

pd.set_option('display.max_rows', None)
pd.options.display.float_format = '{:.4f}'.format
vdf = pd.DataFrame({
    'Variable Name': variable_names,
    'Value': variable_values,
    'Units': variable_units,
    'Description': variable_description
})

print(vdf)

# Handle constants
for con in con_list:
    constant_names.append(con.name)
    constant_values.append(con.value)
    constant_units.append(getattr(con, "_units", None))
    constant_description.append(getattr(con, "doc", None))

pd.set_option('display.max_rows', None)
pd.options.display.float_format = '{:.4}'.format
cdf = pd.DataFrame({
    'Constant Name': constant_names,
    'Value': constant_values,
    'Units': constant_units,
    'Description': constant_description
})

print(cdf)
```

APPENDIX D

PROFILE AND INDUCED DRAG RELATION FOR VARIABLE WAKE FORMULATION

The relationship between profile drag and induced drag is explored by solving the total drag sum as an optimization problem. First, the equivalent aerodynamic relations for D_p and D_i are inserted, with the aspect ratio $AR = \frac{b}{c}$:

$$D = D_p + D_i$$

$$D = \frac{1}{2}\rho_\infty V_\infty^2 c\, b\, C_{D_p} + \frac{1}{2}\rho_\infty V_\infty^2 c\, b\, \frac{C_L^2}{\pi\, e\, AR}$$

$$D = \frac{1}{2}\rho_\infty V_\infty^2 c\, b\, C_{D_p} + \frac{1}{2}\rho_\infty V_\infty^2 c\, b\, \frac{C_L^2}{\pi\, e\, \frac{b}{c}}$$

Plugging in the standard relation $C_L = \frac{L}{\frac{1}{2}\rho_\infty V_\infty^2 c\, b}$ and doing some algebraic manipulation results in

$$D = \frac{1}{2}\rho_\infty V_\infty^2 c\, b\, C_{D_p} + \frac{1}{2}\rho_\infty V_\infty^2 c^2 \frac{\left(\frac{L}{\frac{1}{2}\rho_\infty V_\infty^2 c\, b}\right)^2}{\pi e}$$

$$D = \frac{1}{2}\rho_\infty V_\infty^2 c\, b\, C_{D_p} + \frac{1}{2}\rho_\infty V_\infty^2 \frac{L^2}{\frac{1}{4}\pi e\, \rho_\infty^2 V_\infty^4 b^2}$$

The derivative $\frac{\partial D}{\partial b}$ is used to reflect the induced drag minimization for some span b.

$$\frac{\partial D}{\partial b} = \frac{1}{2}\rho_\infty V_\infty^2 c\, C_{D_p} - 2\left(\frac{1}{2}\rho_\infty V_\infty^2 \frac{L^2}{\frac{1}{4}\pi e\, \rho_\infty^2 V_\infty^4 b^3}\right)$$

Setting $\frac{\partial D}{\partial b}$ to 0 as the minimum drag condition allows the two drag terms to be on opposite sides:

$$\frac{1}{2}\rho_\infty V_\infty^2 c\, C_{D_p} = 2\left(\frac{1}{2}\rho_\infty V_\infty^2 \frac{L^2}{\frac{1}{4}\pi e\, \rho_\infty^2 V_\infty^4 b^3}\right)$$

Multiplying both sides by b and the right side by $\frac{c^2}{c^2}$ allows c_L to be reinserted:

$$\frac{1}{2}\rho_\infty V_\infty^2 c\, b\, C_{D_p} = 2\left(\frac{1}{2}\rho_\infty V_\infty^2 c^2 \frac{L^2}{\frac{1}{4}\pi e\, \rho_\infty^2 V_\infty^4 b^2 c^2}\right)$$

$$\frac{1}{2}\rho_\infty V_\infty^2\, c\, b\, C_{D_p} = 2\left(\frac{1}{2}\rho_\infty V_\infty^2\, c^2 \frac{\left(\frac{L}{\frac{1}{2}\rho_\infty V_\infty^2\, c\, b}\right)^2}{\pi e}\right)$$

$$\frac{1}{2}\rho_\infty V_\infty^2\, c\, b\, C_{D_p} = 2\left(\frac{1}{2}\rho_\infty V_\infty^2\, c^2 \frac{c_L^2}{\pi e}\right)$$

Multiplying the right side by $\frac{b}{b}$ puts the relation back in terms of profile drag and induced drag

$$\frac{1}{2}\rho_\infty V_\infty^2\, c\, b\, C_{D_p} = 2\left(\frac{1}{2}\rho_\infty V_\infty^2\, c\, b \frac{c_L^2}{\pi e \frac{b}{c}}\right)$$

resulting in the final relationship

$$D_p = 2D_i$$

APPENDIX E

INDUCED DRAG TABLES AND SP MODEL RUNTIMES

The induced drags for each panel configuration and model are tabulated, correlating with the induced drag figures from Section 4.2. The Signomial Programming compatible model runtimes are also tabulated against the number of panels in the configuration, as discussed in Section 4.2. The Drela model runtimes did not exceed 10 seconds, did not influence the panel evaluation ranges, and therefore are not tabulated. Finally, the SP compatible model runtimes are plotted against the number of panels for an overall runtime visualization.

TABLE E.1. Induced Drags for Flat Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	188.2128	189.778	0.8283
20	197.1743	197.62	0.225
30	200.3538	200.561	0.1034
40	201.983	202.102	0.0587
50	202.9727	203.049	0.0378
60	203.6384	203.692	0.0261
70	204.1152	204.155	0.0197
80	204.457	204.506	0.024
90	204.7561	204.78	0.0119
100	204.9813	205.001	0.0097

TABLE E.2. Induced Drags for Flat Winglet Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	150.7568	151.719	0.6365
20	155.7783	156.766	0.632
30	157.0462	158.756	1.0831
40	157.7145	159.813	1.3216
50	158.1015	160.468	1.4855
60	158.347	160.914	1.6079

TABLE E.3. SP Model Runtimes for Flat Wake Configurations

Panels	Flat Runtime (s)	Flat + Winglets Runtime (s)
10	1.3	4.2
20	4.5	8.1
30	10.7	54.6
40	21.2	76.3
50	31.7	224.2
60	50.3	555.2
70	91.8	—
80	126.7	—
90	294.3	—
100	286	—

TABLE E.4. Induced Drags for Parabolic Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	183.7187	185.169	0.7863
20	191.9913	192.405	0.2152
30	194.9276	195.12	0.0986
40	196.4327	196.543	0.056
50	197.3472	197.418	0.0361
60	197.9624	198.012	0.025
70	198.4031	198.44	0.0188
80	198.7358	198.765	0.0144
90	198.9955	199.018	0.0114
100	199.2038	199.222	0.0092

TABLE E.5. Induced Drags for Parabolic Winglet Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	146.6488	147.6	0.6463
20	151.6054	152.34	0.4834
30	152.8373	154.199	0.887
40	153.3741	155.19	1.1768
50	153.691	155.805	1.366
60	153.8777	156.224	1.5132

TABLE E.6. SP Model Runtimes for Parabolic Wake Configurations

Panels	Parabolic Runtime (s)	Parabolic + Winglets Runtime (s)
10	1.4	3.4
20	5.7	8.4
30	10.7	81.4
40	18.7	75.7
50	33.2	123.4
60	72.7	589.7
70	101.7	—
80	158.2	—
90	205.1	—
100	299.2	—

TABLE E.7. Induced Drags for Linear Dihedral Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	186.1692	187.72	0.8295
20	194.988	195.393	0.2077
30	198.0705	198.278	0.1049
40	199.6715	199.792	0.0601
50	200.6459	200.724	0.0388
60	201.3013	201.355	0.0269
70	201.7708	201.812	0.02041
80	202.1255	202.157	0.01572
90	202.4023	202.428	0.0125
100	202.6244	202.645	0.01015

TABLE E.8. Induced Drags for Linear Dihedral Winglet Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	148.5068	149.487	0.6578
20	153.5543	154.458	0.5867
30	154.8141	156.412	1.027
40	155.429	157.453	1.2936
50	155.7861	158.099	1.4734
60	156.0067	158.538	1.6098

TABLE E.9. SP Model Runtimes for Linear Dihedral Wake Configurations

Panels	Linear Runtime (s)	Linear + Winglets Runtime (s)
10	1.2	2.6
20	3.9	8.8
30	12.5	70.3
40	20	65.7
50	36.9	176.2
60	60.7	484.5
70	109.1	—
80	174.6	—
90	216.8	—
100	253.4	—

TABLE E.10. Induced Drags for Sine Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	188.0637	189.12	0.5604
20	197.026	196.889	0.0693
30	200.209	199.803	0.2028
40	201.8409	201.33	0.2535
50	202.8327	202.269	0.2781
60	203.5	202.906	0.2924
70	203.978	203.365	0.3008
80	204.3389	203.713	0.3068
90	204.6206	203.985	0.3111
100	204.8466	204.204	0.3144

TABLE E.11. Induced Drags for Sine Winglet Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	150.4613	152.213	1.1576
20	155.5063	157.257	1.1195
30	156.7773	159.248	1.5636
40	157.4456	160.301	1.7975
50	157.8326	160.954	1.9585
60	158.0778	161.399	2.0792

TABLE E.12. SP Model Runtimes for Sine Wake Configurations

Panels	Sine Runtime (s)	Sine + Winglets Runtime (s)
10	1	2.2
20	5.5	11.1
30	8.8	55.3
40	18.4	80.5
50	31.1	211.6
60	51.1	545.8
70	81.4	—
80	122.7	—
90	170.1	—
100	238.8	—

TABLE E.13. Induced Drags for Asymmetric Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	186.3431	188.188	0.9851
20	194.9653	195.594	0.3218
30	198.024	198.364	0.1715
40	199.5907	199.814	0.1116
50	200.5435	200.705	0.0807
60	201.1839	201.309	0.0623
70	201.6426	201.745	0.051
80	201.989	202.075	0.0427
90	202.2593	202.333	0.0365
100	202.476	202.541	0.0319

TABLE E.14. Induced Drags for Asymmetric Winglet Configuration

Panels	D_i (SP)	D_i (Drela)	% Difference
10	166.5531	167.925	0.8204
20	173.2701	173.97	0.403
30	175.343	176.283	0.5349
40	176.3328	177.505	0.6624
50	176.9331	178.259	0.7469
60	177.3192	178.772	0.816

TABLE E.15. SP Model Runtimes for Asymmetric Wake Configurations

Panels	Asymmetric Runtime (s)	Asymmetric + Winglet Runtime (s)
10	1.7	2.1
20	7	6.8
30	10.4	50.4
40	18	76.1
50	31.9	100.2
60	51.6	699.1
70	82.9	—
80	122	—
90	172.1	—
100	240.1	—

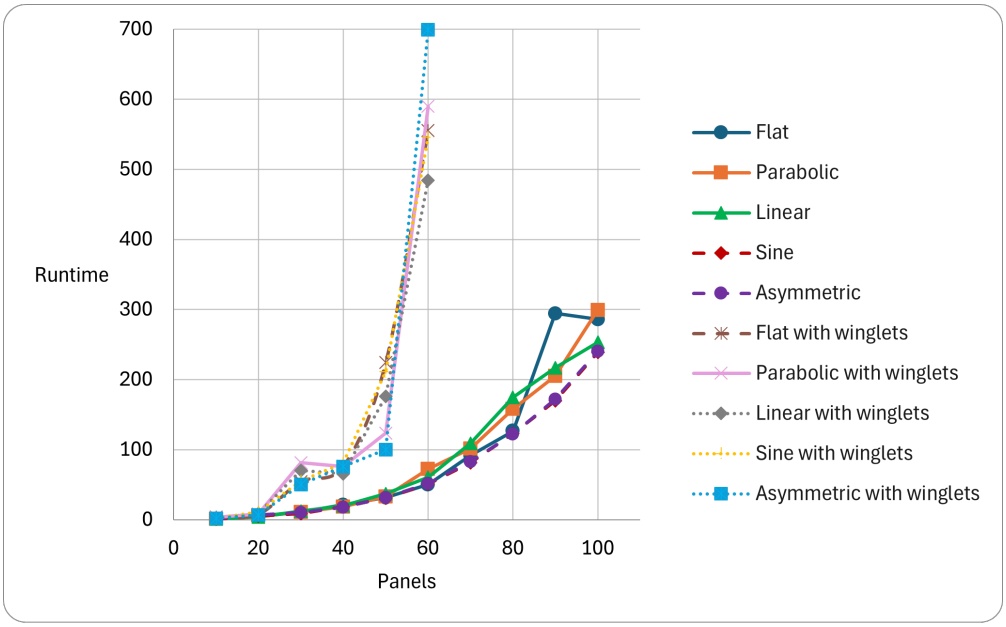

FIGURE E.1. Runtimes for all SP compatible model configurations.

REFERENCES

REFERENCES

[1] Karcher, C. J., *An Optimization Centered Approach To Multifidelity Aircraft Design*, Doctoral Dissertation, Massachusetts Institute of Technology, 2022.

[2] Nocedal, J. and Wright, S. J., *Numerical Optimization*, Springer New York, 2006.

[3] Cramer, E. J., Dennis, Jr., J. E., Frank, P. D., Lewis, R. M., and Shubin, G. R., "Problem Formulation for Multidisciplinary Optimization," *SIAM Journal on Optimization*, Vol. 4, No. 4, November 1994, pp. 754–776.

[4] Martins, J. R. R. A. and Lambe, A. B., "Multidisciplinary Design Optimization: A Survey of Architectures," *AIAA Journal*, Vol. 51, No. 9, September 2013, pp. 2049–2075.

[5] Kirschen, P. G., Burnell, E., and Hoburg, W., "Signomial Programming Models for Aircraft Design," *54th AIAA Aerospace Sciences Meeting*, American Institute of Aeronautics and Astronautics, Reston, Virginia, January 2016, pp. 1–26.

[6] Hoburg, W. and Abbeel, P., "Geometric Programming for Aircraft Design Optimization," *AIAA Journal*, Vol. 52, No. 11, November 2014, pp. 2414–2426.

[7] Duffin, R. J., Peterson, E. L., and Zener, C., *Geometric Programming: Theory and Application*, Wiley, New York, 1967.

[8] Torenbeek, E., *Advanced Aircraft Design: Conceptual Design, Analysis and Optimization of Subsonic Civil Airplanes*, John Wiley & Sons, Ltd, 2013.

[9] Brown, A. and Harris, W., "Vehicle Design and Optimization Model for Urban Air Mobility," *AIAA Journal*, Vol. 57, No. 6, November 2020, pp. 1003–1013.

[10] Boyd, S. P., Kim, S.-J., Patil, D. D., and Horowitz, M. A., "Digital Circuit Optimization Via Geometric Programming," *Operations Research*, Vol. 53, No. 6, December 2005, pp. 899–932.

[11] del Mar Hershenson, M., Boyd, S. P., and Lee, T. H., "Optimal Design of a CMOS Op-Amp Via Geometric Programming," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 20, No. 1, 2001, pp. 1–21.

[12] Jabr, R. A., "Application of Geometric Programming To Transformer Design," *IEEE Transactions on Magnetics*, Vol. 41, No. 11, November 2005, pp. 4261–4269.

[13] Hoburg, W. and Abbeel, P., "Fast Wind Turbine Design Via Geometric Programming," *54th AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*, American Institute of Aeronautics and Astronautics, Reston, Virginia, April 2013, pp. 1–9.

[14] Marin-Sanguino, A., Voit, E. O., Gonzalez-Alcon, C., and Torres, N. V., "Optimization of Biotechnological Systems Through Geometric Programming." *Theoretical Biology & Medical Modelling*, Vol. 4, September 2007, pp. 38.

[15] Sela Perelman, L. and Amin, S., "Control of Tree Water Networks: A Geometric Programming Approach," *Water Resources Research*, Vol. 51, No. 10, October 2015, pp. 8409–8430.

[16] Vera, J., González-Alcón, C., Marín-Sanguino, A., and Torres, N., "Optimization of Biochemical Systems Through Mathematical Programming: Methods and Applications," *Computers & Operations Research*, Vol. 37, No. 8, August 2010, pp. 1427–1438.

[17] Misra, S., Fisher, M. W., Backhaus, S., Bent, R., Chertkov, M., and Pan, F., "Optimal Compression in Natural Gas Networks: A Geometric Programming Approach," *IEEE Transactions on Control of Network Systems*, Vol. 2, No. 1, March 2015, pp. 47–56.

[18] Boyd, S., Kim, S.-J., Vandenberghe, L., and Hassibi, A., "A Tutorial on Geometric Programming," *Optimization and Engineering*, Vol. 8, No. 1, May 2007, pp. 67–127.

[19] Boyd, S. P. and Vandenberghe, L., *Convex Optimization*, Cambridge University Press, 7th ed., 2009.

[20] Andersen, M., Dahl, J., Liu, Z., and Vandenberghe, L., "Interior-Point Methods for Large-Scale Cone Programming," *Optimization for Machine Learning*, edited by S. Sra, S. Nowozin, and S. J. Wright, The MIT Press, 2012, pp. 55–84.

[21] York, M. A., Hoburg, W. W., and Drela, M., "Turbofan Engine Sizing and Tradeoff Analysis Via Signomial Programming," *Journal of Aircraft*, Vol. 55, No. 3, May 2018, pp. 988–1003.

[22] York, M. A., Öztürk, B., Burnell, E., and Hoburg, W. W., "Efficient Aircraft Multidisciplinary Design Optimization and Sensitivity Analysis Via Signomial Programming," *AIAA Journal*, Vol. 56, No. 11, November 2018, pp. 4546–4561.

[23] Gilmore, C. K., Barrett, S. R. H., Brown, A., and Xu, H., "Solid-State Electroaerodynamic Aircraft Design Using Signomial Programming," *AIAA Journal*, Vol. 61, No. 1, January 2024, pp. 1–11.

[24] Kirschen, P. G. and Hoburg, W. W., "The Power of Log Transformation: A Comparison of Geometric and Signomial Programming with General Nonlinear Programming Techniques for Aircraft Design Optimization," *2018 AIAA/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*, American Institute of Aeronautics and Astronautics, Reston, Virginia, January 2018, pp. 1–23.

[25] Hall, D. K., Dowdle, A., Gonzalez, J., Trollinger, L., and Thalheimer, W., "Assessment of a Boundary Layer Ingesting Turboelectric Aircraft Configuration Using Signomial Programming," *2018 Aviation Technology, Integration, and Operations Conference*, American Institute of Aeronautics and Astronautics, Reston, Virginia, June 2018, pp. 1–16.

[26] Ozturk, B. and Saab, A., "Optimal Aircraft Design Decisions Under Uncertainty Using Robust Signomial Programming," *AIAA Journal*, Vol. 59, No. 5, May 2021, pp. 1747–1760.

[27] Lin, B., Carpenter, M., and de Weck, O., "Simultaneous Vehicle and Trajectory Design Using Convex Optimization," *AIAA Scitech 2020 Forum*, American Institute of Aeronautics and Astronautics, Reston, Virginia, January 2020, pp. 1–18.

[28] Lipp, T. and Boyd, S., "Variations and Extension of the Convex–concave Procedure," *Optimization and Engineering*, Vol. 17, No. 2, June 2016, pp. 263–287.

[29] Karcher, C. J., "Data Fitting with Signomial Programming Compatible Difference of Convex Functions," *Optimization and Engineering*, April 2022.

[30] Burnell, E., Damen, N. B., and Hoburg, W., "GPkit: A Human-Centered Approach To Convex Optimization in Engineering Design," *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, ACM, New York, NY, USA, April 2020, pp. 1–13.

[31] Hart, W. E., Laird, C. D., Watson, J.-P., Woodruff, D. L., Hackebeil, G. A., Nicholson, B. L., and Siirola, J. D., Pyomo – Optimization Modeling in Python, Vol. 67 of *Springer Optimization and Its Applications*, Springer, 2nd ed., 2017.

[32] Hunter, J. D., "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, Vol. 9, No. 3, May 2007, pp. 90–95.

[33] McKinney, W., "Data Structures for Statistical Computing in Python," *Proceedings of the 9th Python in Science Conference*, Vol. 445, 2010, pp. 56–61.

[34] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al., "Array Programming with NumPy," *Nature*, Vol. 585, No. 7825, September 2020, pp. 357–362.

[35] Prandtl, L., "Tragflügeltheorie," *Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse*, 1918, pp. 451–477.

[36] Drela, M., *Flight Vehicle Aerodynamics*, The MIT Press, 2014.

[37] Anderson, J. D., *Fundementals of Aerodynamics*, McGraw Hill, New York, 5th ed., 2011.

[38] Gudmundsson, S., "The Anatomy of the Wing," *General Aviation Aircraft Design*, edited by S. Gudmundsson, Elsevier, Amsterdam, The Netherlands, 2022, pp. 321–413.

[39] Leishman, J. G., "Lifting Line Theory," *Introduction to Aerospace Flight Vehicles*, Embry-Riddle Aeronautical University, Daytona Beach, Florida, 2022.

[40] Cessna Aircraft Company, *Pilot's Operating Handbook and FAA Approved Airplane Flight Manual: Cessna Model 172S*, Cessna Aircraft Company, Wichita, Kansas, 1998, Original issue July 8, 1998.

[41] Whitcomb, R. T., "A Design Approach and Selected Wind-Tunnel Results at High Subsonic Speeds for Wing-Tip Mounted Winglets," Tech. Rep. NASA TN D-8260, National Aeronautics and Space Administration, Washington, DC, July 1976.

[42] Drela, M., *TASOPT: Technical Reference and User's Manual*, Massachusetts Institute of Technology (MIT), 2011, http://web.mit.edu/drela/Public/web/tasopt/.

[43] Avila, D. B., *Aircraft Design Optimization Using Signomial Programming*, Master's Thesis, California State University, Long Beach, 2025.

[44] Ning, S. A. and Kroo, I., "Tip Extensions, Winglets, and C-Wings: Conceptual Design and Optimization," *26th AIAA Applied Aerodynamics Conference*, American Institute of Aeronautics and Astronautics, Honolulu, Hawaii, August 2008, pp. 1–17.

[45] Martins, J. R. R. A. and Lambe, A. B., "Multidisciplinary Design Optimization: A Survey of Architectures," *AIAA Journal*, Vol. 51, No. 9, 2013, pp. 2049–2075.

[46] Kroo, I., Altus, S., Braun, R., Gage, P., and Sobieski, I., "Multidisciplinary Optimization Methods for Aircraft Preliminary Design," Tech. rep., Stanford University, Stanford, California, 1994, Presented at the 5th AIAA/USAF/NASA/ISSMO Symposium on Multidisciplinary Analysis and Optimization.

[47] Hoburg, W. W. and Abbeel, P., "Geometric Programming for Aircraft Design Optimization," *52nd Aerospace Sciences Meeting*, AIAA, National Harbor, MD, 2014.

[48] Kirschen, P. G., York, M. A., Ozturk, B., and Hoburg, W. W., "Application of Signomial Programming to Aircraft Design," *Journal of Aircraft*, Vol. 56, No. 1, 2019, pp. 389–403.

[49] Boyd, S., Kim, S.-J., Vandenberghe, L., and Hassibi, A., "A Tutorial on Geometric Programming," *Optimization and Engineering*, Vol. 8, No. 1, 2007, pp. 67–127.

[50] Hart, W. E., *Pyomo: Python Optimization Modeling Objects*, 2022, https://pyomo.readthedocs.io/.